

LoRA: LOW-RANK ADAPTATION OF LARGE LANGUAGE MODELS

Edward Hu* Yelong Shen* Phillip Wallis Zeyuan Allen-Zhu
Yuanzhi Li Shean Wang Lu Wang Weizhu Chen
Microsoft Corporation
{edwardhu, yeshe, phwallis, zeyuana,
yuanzhil, swang, luw, wzchen}@microsoft.com
yuanzhil@andrew.cmu.edu
(Version 2)

ABSTRACT

An important paradigm of natural language processing consists of large-scale pre-training on general domain data and adaptation to particular tasks or domains. As we pre-train larger models, full fine-tuning, which retraining all model parameters, becomes less feasible. Using GPT-3 175B as an example – deploying independent instances of fine-tuned models, each with 175B parameters, is prohibitively expensive. We propose **Low-Rank Adaptation**, or LoRA, which freezes the pre-trained model weights and injects trainable rank decomposition matrices into each layer of the Transformer architecture, greatly reducing the number of trainable parameters for downstream tasks. Compared to GPT-3 175B fine-tuned with Adam, LoRA can reduce the number of trainable parameters by 10,000 times and the GPU memory requirement by 3 times. LoRA performs on-par or better than fine-tuning in model quality on RoBERTa, DeBERTa, GPT-2, and GPT-3, despite having fewer trainable parameters, a higher training throughput, and, unlike adapters, *no additional inference latency*. We also provide an empirical investigation into rank-deficiency in language model adaptation, which sheds light on the efficacy of LoRA. We release a package that facilitates the integration of LoRA with PyTorch models and provide our implementations and model checkpoints for RoBERTa, DeBERTa, and GPT-2 at <https://github.com/microsoft/LoRA>.

1 INTRODUCTION

Many applications in natural language processing rely on adapting *one* large-scale, pre-trained language model to *multiple* downstream applications. Such adaptation is usually done via *fine-tuning*, which updates all the parameters of the pre-trained model. The major downside of fine-tuning is that the new model contains as many parameters as in the original model. As larger models are trained every few months, this changes from a mere “inconvenience” for GPT-2 (Radford et al., b) or RoBERTa large (Liu et al., 2019) to a critical deployment challenge for GPT-3 (Brown et al., 2020) with 175 billion trainable parameters.¹

Many sought to mitigate this by adapting only some parameters or learning external modules for new tasks. This way, we only need to store and load a small number of task-specific parameters in addition to the pre-trained model for each task, greatly boosting the operational efficiency when deployed. However, existing techniques

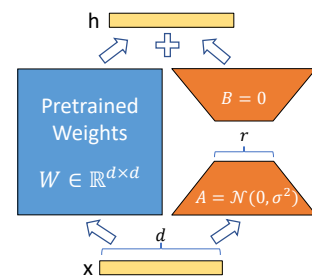


Figure 1: Our reparametrization. We only train A and B .

*Equal contribution.

⁰Compared to V1, this draft includes better baselines, experiments on GLUE, and more on adapter latency.

¹While GPT-3 175B achieves non-trivial performance with few-shot learning, fine-tuning boosts its performance significantly as shown in Appendix A.

LoRA: 大型语言模型的低秩适应

Edward Hu* Yelong Shen* Phillip Wallis Zeyuan Allen-Zhu
Yuanzhi Li Shean Wang Lu Wang Weizhu Chen
Microsoft Corporation{edwardhu, yeshe, phwallis,
zeyuana, yuanzhil, swang, luw, wzchen}
@microsoft.com yuanzhil@andrew.cmu.edu (版本 2)

摘要

自然语言处理的一个重要范式包括在通用领域数据上进行大规模预训练，并适应特定的任务或领域。随着我们预训练更大的模型，全微调（即重新训练所有模型参数）变得不再可行。以 GPT-3 175B 为例——部署具有 175B 参数的独立微调模型实例，成本极其高昂。我们提出了 **低秩适应**，即 LoRA，该方法冻结预训练模型的权重，并将可训练的秩分解矩阵注入 Transformer 架构的每一层，极大地减少了下游任务的可训练参数数量。与使用 Adam 微调的 GPT-3 175B 相比，LoRA 可将可训练参数数量减少 10,000 倍，并将 GPU 内存需求降低 3 倍。尽管可训练参数更少，训练吞吐量更高，且与适配器不同，不会增加额外的推理延迟，LoRA 在 RoBERTa、DeBERTa、GPT-2 和 GPT-3 上的模型质量表现相当或更优。我们还对语言模型适应中的秩亏问题进行了实证研究，这为 LoRA 的有效性提供了依据。我们发布了一个包，以促进 LoRA 与 PyTorch 模型的集成，并在 <https://github.com/microsoft/LoRA> 上提供了我们的实现和 RoBERTa、DeBERTa 和 GPT-2 的模型检查点。

1 简介

许多自然语言处理应用依赖于将一个大型预训练语言模型适应多个下游应用。这种适应通常通过微调完成，它会更新预训练模型的所有参数。微调的主要缺点是，新模型包含与原始模型一样多的参数。随着更大型的模型每隔几个月就进行训练，这从 GPT-2 (Radford 等人, b) 或 RoBERTa large (Liu 等人, 2019) 的“不便”变成了 GPT-3 (Brown 等人, 2020) 的关键部署挑战，后者有 1750 亿个可训练参数。¹

许多人试图通过仅调整部分参数或为新任务学习外部模块来缓解这一问题。这样，我们只需在预训练模型之外，为每个任务存储和加载少量特定于任务的参数，从而在部署时大幅提升操作效率。然而，现有技术

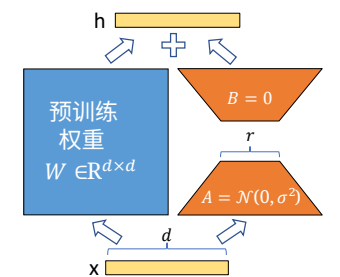


图1：我们的重新参数化。我们只训练 A 和 B 。

*同等贡献。⁰与 V1 相比，这份草稿包括更好的基线、GLUE 上的实验以及更多关于适配器延迟的内容。¹虽然 GPT-3 175B 通过少样本学习实现了不平凡的性能，但微调显著提升了其性能，如附录 A 所示。

often introduce inference latency (Houlsby et al., 2019; Rebuffi et al., 2017) by extending model depth or reduce the model’s usable sequence length (Li & Liang, 2021; Lester et al., 2021; Ham-bardzumyan et al., 2020; Liu et al., 2021) (Section 3). More importantly, these method often fail to match the fine-tuning baselines, posing a trade-off between efficiency and model quality.

We take inspiration from Li et al. (2018a); Aghajanyan et al. (2020) which show that the learned over-parametrized models in fact reside on a low intrinsic dimension. We hypothesize that the change in weights during model adaptation also has a low “intrinsic rank”, leading to our proposed **Low-Rank Adaptation (LoRA)** approach. LoRA allows us to train some dense layers in a neural network indirectly by optimizing rank decomposition matrices of the dense layers’ change during adaptation instead, while keeping the pre-trained weights frozen, as shown in Figure 1. Using GPT-3 175B as an example, we show that a very low rank (i.e., r in Figure 1 can be one or two) suffices even when the full rank (i.e., d) is as high as 12,288, making LoRA both storage- and compute-efficient.

LoRA possesses several key advantages.

- A pre-trained model can be shared and used to build many small LoRA modules for different tasks. We can freeze the shared model and efficiently switch tasks by replacing the matrices A and B in Figure 1, reducing the storage requirement and task-switching overhead significantly.
- LoRA makes training more efficient and lowers the hardware barrier to entry by up to 3 times when using adaptive optimizers since we do not need to calculate the gradients or maintain the optimizer states for most parameters. Instead, we only optimize the injected, much smaller low-rank matrices.
- Our simple linear design allows us to merge the trainable matrices with the frozen weights when deployed, *introducing no inference latency* compared to a fully fine-tuned model, by construction.
- LoRA is orthogonal to many prior methods and can be combined with many of them, such as prefix-tuning. We provide an example in Appendix E.

Terminologies and Conventions We make frequent references to the Transformer architecture and use the conventional terminologies for its dimensions. We call the input and output dimension size of a Transformer layer d_{model} . We use W_q , W_k , W_v , and W_o to refer to the query/key/value/output projection matrices in the self-attention module. W or W_0 refers to a pre-trained weight matrix and ΔW its accumulated gradient update during adaptation. We use r to denote the rank of a LoRA module. We follow the conventions set out by (Vaswani et al., 2017; Brown et al., 2020) and use Adam (Loshchilov & Hutter, 2019; Kingma & Ba, 2017) for model optimization and use a Transformer MLP feedforward dimension $d_{ffn} = 4 \times d_{model}$.

2 PROBLEM STATEMENT

While our proposal is agnostic to training objective, we focus on language modeling as our motivating use case. Below is a brief description of the language modeling problem and, in particular, the maximization of conditional probabilities given a task-specific prompt.

Suppose we are given a pre-trained autoregressive language model $P_\Phi(y|x)$ parametrized by Φ . For instance, $P_\Phi(y|x)$ can be a generic multi-task learner such as GPT (Radford et al., b; Brown et al., 2020) based on the Transformer architecture (Vaswani et al., 2017). Consider adapting this pre-trained model to downstream conditional text generation tasks, such as summarization, machine reading comprehension (MRC), and natural language to SQL (NL2SQL). Each downstream task is represented by a training dataset of context-target pairs: $\mathcal{Z} = \{(x_i, y_i)\}_{i=1, \dots, N}$, where both x_i and y_i are sequences of tokens. For example, in NL2SQL, x_i is a natural language query and y_i its corresponding SQL command; for summarization, x_i is the content of an article and y_i its summary.

通常通过增加模型深度来引入推理延迟 (Houlsby等人, 2019; Rebuffi等人, 2017), 或减少模型可用的序列长度 (李&梁, 2021; Lester等人, 2021; Hammandzumyan等人, 2020; Liu等人, 2021) (第3节)。更重要的是, 这些方法往往无法达到微调基线, 从而在效率和模型质量之间造成权衡。

我们从 Li 等人 (2018a); Aghajanyan 等人 (2020) 的研究中获得启发, 这些研究显示, 所学习的过参数化模型实际上位于一个低内在维度的空间上。我们假设, 在模型适应过程中权重的变化也具有一个低的“内在秩”, 这导致我们提出了**Low-Rank Adaptation (LoRA)**方法。LoRA 允许我们通过优化密集层在适应过程中权重变化时的秩分解矩阵, 而不是直接训练神经网络中的某些密集层, 同时保持预训练权重冻结, 如图 1 所示。以 GPT-3 175B 为例, 我们展示了即使全秩 (即 d) 高达 12,288 时, 一个非常低的秩 (即图 1 中的 r , 可以是一个或两个) 就足够了, 这使得 LoRA 既节省存储空间又高效计算。

LoRA 具有以下几个关键优势。

- 一个预训练模型可以被共享, 并用于为不同的任务构建许多小的 LoRA 模块。我们可以冻结共享模型, 并通过替换图 1 中的 A 和 B 矩阵来高效地切换任务, 从而显著降低存储需求和任务切换的开销。
- LoRA 通过使用自适应优化器使训练更高效, 并通过高达3倍的提升降低了硬件门槛, 因为我们不需要为大多数参数计算梯度或维护优化器状态。相反, 我们只需优化注入的、远小于原矩阵的低秩矩阵。
- 我们简单的线性设计使我们能够在部署时将可训练矩阵与冻结权重合并, 与完全微调的模型相比, 构造上不会引入推理延迟。
- LoRA 与许多先前方法正交, 并且可以与其中许多方法结合使用, 例如前缀调优。我们在附录E中提供了一个示例。

术语和约定 我们频繁引用 Transformer 架构, 并使用其维度的传统术语。我们称 Transformer 层的输入和输出维度大小为 d_{model} 。我们使用 W_q , W_k , W_v 和 W_o 来指代自注意力模块中的查询/键/值/输出投影矩阵。 W 或 W_0 指代预训练的权重矩阵, ΔW 及其在适应过程中累积的梯度更新。我们使用 r 表示 LoRA 模块的秩。我们遵循 (Vaswani 等人, 2017年; 布朗等人, 2020年) 设定的约定, 并使用 Adam (Loshchilov & Hutter, 2019年; Kingma & Ba, 2017年) 进行模型优化, 并使用 Transformer MLP 前馈维度 $d_{ffn} = 4 \times d_{model}$ 。

2 问题陈述

虽然我们的提案对训练目标保持中立, 但我们以语言建模作为主要的应用场景。以下是语言建模问题的简要描述, 特别是针对特定任务提示的条件概率最大化。

假设我们给定一个由 Φ 参数化的预训练自回归语言模型 $P_\Phi(y|x)$ 。例如, $P_\Phi(y|x)$ 可以是基于 Transformer 架构 (Vaswani 等人, 2017) 的通用多任务学习器, 如 GPT (Radford 等人, b; 布朗等人, 2020)。考虑将此预训练模型适配到下游条件文本生成任务, 例如摘要、机器阅读理解 (MRC) 和自然语言到 SQL (NL2SQL)。每个下游任务由一个上下文-目标对的数据集表示: $\mathcal{Z} = \{(x_i, y_i)\}_{i=1, \dots, N}$, 其中 x_i 和 y_i 都是标记序列。例如, 在 NL2SQL 中, x_i 是一个自然语言查询, y_i 是其对应的 SQL命令; 对于摘要, x_i 是一篇文章的内容, y_i 是其摘要。

During full fine-tuning, the model is initialized to pre-trained weights Φ_0 and updated to $\Phi_0 + \Delta\Phi$ by repeatedly following the gradient to maximize the conditional language modeling objective:

$$\max_{\Phi} \sum_{(x,y) \in \mathcal{Z}} \sum_{t=1}^{|y|} \log (P_{\Phi}(y_t|x, y_{<t})) \quad (1)$$

One of the main drawbacks for full fine-tuning is that for *each* downstream task, we learn a *different* set of parameters $\Delta\Phi$ whose dimension $|\Delta\Phi|$ equals $|\Phi_0|$. Thus, if the pre-trained model is large (such as GPT-3 with $|\Phi_0| \approx 175$ Billion), storing and deploying many independent instances of fine-tuned models can be challenging, if at all feasible.

In this paper, we adopt a more parameter-efficient approach, where the task-specific parameter increment $\Delta\Phi = \Delta\Phi(\Theta)$ is further encoded by a much smaller-sized set of parameters Θ with $|\Theta| \ll |\Phi_0|$. The task of finding $\Delta\Phi$ thus becomes optimizing over Θ :

$$\max_{\Theta} \sum_{(x,y) \in \mathcal{Z}} \sum_{t=1}^{|y|} \log (p_{\Phi_0 + \Delta\Phi(\Theta)}(y_t|x, y_{<t})) \quad (2)$$

In the subsequent sections, we propose to use a low-rank representation to encode $\Delta\Phi$ that is both compute- and memory-efficient. When the pre-trained model is GPT-3 175B, the number of trainable parameters $|\Theta|$ can be as small as 0.01% of $|\Phi_0|$.

3 AREN’T EXISTING SOLUTIONS GOOD ENOUGH?

The problem we set out to tackle is by no means new. Since the inception of transfer learning, dozens of works have sought to make model adaptation more parameter- and compute-efficient. See Section 6 for a survey of some of the well-known works. Using language modeling as an example, there are two prominent strategies when it comes to efficient adaptations: adding adapter layers (Houlsby et al., 2019; Rebuffi et al., 2017; Pfeiffer et al., 2021; Rüklé et al., 2020) or optimizing some forms of the input layer activations (Li & Liang, 2021; Lester et al., 2021; Hambardzumyan et al., 2020; Liu et al., 2021). However, both strategies have their limitations, especially in a large-scale and latency-sensitive production scenario.

Adapter Layers Introduce Inference Latency There are many variants of adapters. We focus on the original design by Houlsby et al. (2019) which has two adapter layers per Transformer block and a more recent one by Lin et al. (2020) which has only one per block but with an additional LayerNorm (Ba et al., 2016). While one can reduce the overall latency by pruning layers or exploiting multi-task settings (Rüklé et al., 2020; Pfeiffer et al., 2021), there is no direct ways to bypass the extra compute in adapter layers. This seems like a non-issue since adapter layers are designed to have few parameters (sometimes <1% of the original model) by having a small bottleneck dimension, which limits the FLOPs they can add. However, large neural networks rely on hardware parallelism to keep the latency low, and adapter layers have to be processed sequentially. This makes a difference in the online inference setting where the batch size is typically as small as one. In a generic scenario without model parallelism, such as running inference on GPT-2 (Radford et al., b) medium on a single GPU, we see a noticeable increase in latency when using adapters, even with a very small bottleneck dimension (Table 1).

This problem gets worse when we need to shard the model as done in Shoeybi et al. (2020); Lepikhin et al. (2020), because the additional depth requires more synchronous GPU operations such as AllReduce and Broadcast, unless we store the adapter parameters redundantly many times.

Directly Optimizing the Prompt is Hard The other direction, as exemplified by prefix tuning (Li & Liang, 2021), faces a different challenge. We observe that prefix tuning is difficult to optimize and that its performance changes non-monotonically in trainable parameters, confirming similar observations in the original paper. More fundamentally, reserving a part of the sequence length for adaptation necessarily reduces the sequence length available to process a downstream task, which we suspect makes tuning the prompt less performant compared to other methods. We defer the study on task performance to Section 5.

在**全微调**期间，模型被初始化为**预训练权重** Φ_0 并更新为 $\Phi_0 + \Delta\Phi$ ，通过反复沿着梯度方向优化，以最大化条件**语言建模**目标：

$$\max_{\Phi} \sum_{(x,y) \in \mathcal{Z}} \sum_{t=1}^{|y|} \log (P_{\Phi}(y_t|x, y_{<t})) \quad (1)$$

全微调的主要缺点之一在于，对于<样式 id='1'>每个</样式>下游任务，我们学习一组<样式 id='3'>不同的</样式>参数 $\Delta\Phi$ ，其维度 $|\Delta\Phi|$ 等于 $|\Phi_0|$ 。因此，如果预训练模型很大（例如拥有 $|\Phi_0| \approx 175$ 十亿的GPT-3），存储和部署许多独立的微调模型实例可能会具有挑战性，甚至可能不可行。

在本文中，我们采用了一种更参数高效的策略，其中特定任务的参数增量 $\Delta\Phi = \Delta\Phi(\Theta)$ 由一个尺寸远小的参数集 Θ 进一步编码，该参数集具有 $|\Theta| \ll |\Phi_0|$ 。因此，寻找 $\Delta\Phi$ 的任务就变成了在 Θ 上进行优化：

$$\max_{\Theta} \sum_{(x,y) \in \mathcal{Z}} \sum_{t=1}^{|y|} \log (p_{\Phi_0 + \Delta\Phi(\Theta)}(y_t|x, y_{<t})) \quad (2)$$

在后续章节中，我们提出使用低秩表示来编码 $\Delta\Phi$ ，这种表示在计算和内存效率上都很高。当预训练模型为GPT-3 175B时，可训练参数的数量 $|\Theta|$ 可以小到0<样式 id='3'>.</样式>01%的 $|\Phi_0|$ 。

3 个现有解决方案还不够好？

我们试图解决的问题绝非新问题。自迁移学习出现以来，已有数十项工作试图使模型适应更加参数和计算高效。参见第6节，了解一些著名工作的综述。以语言建模为例，在高效适应方面有两种突出策略：添加适配器层（Houlsby等人，2019；Rebuffi等人，2017；Pfeiffer等人，2021；Rüklé等人，2020）或优化某些输入层激活形式（李&梁，2021；Lester等人，2021；Hambardzumyan等人，2020；刘等人，2021）。然而，这两种策略都有其局限性，尤其是在大规模且对延迟敏感的生产场景中。

适配器层引入推理延迟 适配器有许多变体。我们关注Houlsby等人（2019）提出的原始设计，该设计每个Transformer模块有两个适配器层，以及Lin等人（2020）提出的一个较新的设计，每个模块只有一个适配器层，但有一个额外的LayerNorm（Ba等人，2016）。虽然可以通过剪枝层或利用多任务设置（Rüklé等人，2020；Pfeiffer等人，2021）来降低整体延迟，但没有直接方法可以绕过适配器层的额外计算。这似乎不是一个问题，因为适配器层通过具有较小的瓶颈维度被设计为具有很少的参数（有时是 <1%的原始模型），这限制了它们可以添加的FLOPs。然而，大型神经网络依赖于硬件并行性来保持低延迟，而适配器层必须按顺序处理。这在在线推理设置中有所不同，其中批量大小通常为1。在没有模型并行性的通用场景中，例如在单个GPU上对GPT-2（Radford等人，b）中等的推理，当使用适配器时，即使瓶颈维度非常小，我们也会看到明显的延迟增加（表1）。

当我们需要像Shoeybi等人（2020）；Lepikhin等人（2020）那样对模型进行分片时，这个问题会更严重；因为额外的深度需要更多的同步GPU操作，例如AllReduce和Broadcast，除非我们将适配器参数冗余地存储多次。

直接优化提示词很困难 另一个方向，以前缀微调（李 & 梁，2021）为例，面临不同的挑战。我们观察到前缀微调难以优化，且其性能在可训练参数上的变化是非单调的，这与原始论文中的类似观察结果一致。更根本地，为适应预留部分序列长度必然会减少用于处理下游任务的序列长度，我们推测这导致提示词调优的性能不如其他方法。我们将任务性能的研究推迟到第5节。

Batch Size	32	16	1
Sequence Length	512	256	128
$ \Theta $	0.5M	11M	11M
Fine-Tune/LoRA	1449.4±0.8	338.0±0.6	19.8±2.7
Adapter ^L	1482.0±1.0 (+2.2%)	354.8±0.5 (+5.0%)	23.9±2.1 (+20.7%)
Adapter ^H	1492.2±1.0 (+3.0%)	366.3±0.5 (+8.4%)	25.8±2.2 (+30.3%)

Table 1: Inference latency of a single forward pass in GPT-2 medium measured in milliseconds, averaged over 100 trials. We use an NVIDIA Quadro RTX8000. “ $|\Theta|$ ” denotes the number of trainable parameters in adapter layers. Adapter^L and Adapter^H are two variants of adapter tuning, which we describe in Section 5.1. The inference latency introduced by adapter layers can be significant in an online, short-sequence-length scenario. See the full study in Appendix B.

4 OUR METHOD

We describe the simple design of LoRA and its practical benefits. The principles outlined here apply to any dense layers in deep learning models, though we only focus on certain weights in Transformer language models in our experiments as the motivating use case.

4.1 LOW-RANK-PARAMETRIZED UPDATE MATRICES

A neural network contains many dense layers which perform matrix multiplication. The weight matrices in these layers typically have full-rank. When adapting to a specific task, Aghajanyan et al. (2020) shows that the pre-trained language models have a low “intrinsic dimension” and can still learn efficiently despite a random projection to a smaller subspace. Inspired by this, we hypothesize the updates to the weights also have a low “intrinsic rank” during adaptation. For a pre-trained weight matrix $W_0 \in \mathbb{R}^{d \times k}$, we constrain its update by representing the latter with a low-rank decomposition $W_0 + \Delta W = W_0 + BA$, where $B \in \mathbb{R}^{d \times r}$, $A \in \mathbb{R}^{r \times k}$, and the rank $r \ll \min(d, k)$. During training, W_0 is frozen and does not receive gradient updates, while A and B contain trainable parameters. Note both W_0 and $\Delta W = BA$ are multiplied with the same input, and their respective output vectors are summed coordinate-wise. For $h = W_0x$, our modified forward pass yields:

$$h = W_0x + \Delta Wx = W_0x + BAx \quad (3)$$

We illustrate our reparametrization in Figure 1. We use a random Gaussian initialization for A and zero for B , so $\Delta W = BA$ is zero at the beginning of training. We then scale ΔWx by $\frac{\alpha}{r}$, where α is a constant in r . When optimizing with Adam, tuning α is roughly the same as tuning the learning rate if we scale the initialization appropriately. As a result, we simply set α to the first r we try and do not tune it. This scaling helps to reduce the need to retune hyperparameters when we vary r (Yang & Hu, 2021).

A Generalization of Full Fine-tuning. A more general form of fine-tuning allows the training of a subset of the pre-trained parameters. LoRA takes a step further and does not require the accumulated gradient update to weight matrices to have full-rank during adaptation. This means that when applying LoRA to all weight matrices and training all biases², we roughly recover the expressiveness of full fine-tuning by setting the LoRA rank r to the rank of the pre-trained weight matrices. In other words, as we increase the number of trainable parameters³, training LoRA roughly converges to training the original model, while adapter-based methods converges to an MLP and prefix-based methods to a model that cannot take long input sequences.

No Additional Inference Latency. When deployed in production, we can explicitly compute and store $W = W_0 + BA$ and perform inference as usual. Note that both W_0 and BA are in $\mathbb{R}^{d \times k}$. When we need to switch to another downstream task, we can recover W_0 by subtracting BA and then adding a different $B'A'$, a quick operation with very little memory overhead. Critically, this

²They represent a negligible number of parameters compared to weights.

³An inevitability when adapting to hard tasks.

批大小	32	16	1
序列长度	512	256	128
$ \Theta $	0.5M	11M	11M
微调/LoRA	1449.4±0.8	338.0±0.6	19.8±2.7
适配器 ^L	1482.0±1.0 (+2.2%)	354.8±0.5 (+5.0%)	23.9±2.1 (+20.7%)
适配器 ^H	1492.2±1.0 (+3.0%)	366.3±0.5 (+8.4%)	25.8±2.2 (+30.3%)

表1: GPT-2中单次前向传递的推理延迟，以毫秒为单位，在100次试验中取平均值。我们使用NVIDIA Quadro RTX8000。“ $|\Theta|$ ”表示适配器层中的可训练参数数量。Adapter^L和Adapter^H是适配器调整的两个变体，我们在第5.1节中进行了描述。适配器层引入的推理延迟在一个在线、短序列长度的场景中可能很显著。请参阅附录B中的完整研究。

4 我们的方法

我们描述了 LoRA 的简单设计及其实际效益。这里概述的原则适用于深度学习模型中的任何密集层，尽管在我们的实验中，我们仅关注 Transformer 语言模型中的某些权重，因为这是主要的用例。

4.1 低秩参数化更新矩阵

神经网络包含许多执行矩阵乘法的密集层。这些层的权重矩阵通常具有满秩。当适应特定任务时，Aghajanyan 等人 (2020) 指出，预训练语言模型具有低“内在维度”，并且尽管被随机投影到更小的子空间，仍然可以高效学习。受此启发，我们假设在适应过程中权重的更新也具有低“内在秩”。对于预训练权重矩阵 $W_0 \in \mathbb{R}^{d \times k}$ ，我们通过用低秩分解 $W_0 + \Delta W = W_0 + BA$ 表示其更新来约束它，其中 $B \in \mathbb{R}^{d \times r}$, $A \in \mathbb{R}^{r \times k}$ ，以及秩 $r \ll \min(d, k)$ 。在训练过程中， W_0 被冻结并且不接收梯度更新，而 A 和 B 包含可训练参数。请注意， W_0 和 $\Delta W = BA$ 都与相同的输入相乘，并且它们的各自输出向量按坐标相加。对于 $h = W_0x$ ，我们修改后的前向传递产生：

$$h = W_0x + \Delta Wx = W_0x + BAx \quad (3)$$

我们在图1中说明了我们的重新参数化方法。我们使用随机高斯初始化来初始化 A ，使用零初始化 B ，因此 $\Delta W = BA$ 在训练开始时为零。然后我们将 ΔWx 乘以 $\frac{\alpha}{r}$ ，其中 α 是 r 中的一个常数。在使用Adam进行优化时，如果我们适当缩放初始化，调整 α 大致与调整学习率相同。因此，我们只需将 α 设置为第一个尝试的 r ，并且不对其进行调整。这种缩放有助于减少在变化 r 时重新调整超参数的需求 (Yang & Hu, 2021)。

全微调的推广。 一种更通用的微调形式允许训练预训练参数的子集。LoRA更进一步，在适应过程中不需要累积的梯度更新权重矩阵具有满秩。这意味着当将LoRA应用于所有权重矩阵并训练所有偏差²时，通过将LoRA秩 r 设置为预训练权重矩阵的秩，我们大致恢复了全微调的表达能力。换句话说，随着可训练参数数量的增加，训练LoRA大致收敛到训练原始模型，而基于适配器的方法收敛到MLP，基于前缀的方法收敛到无法处理长输入序列的模型。

无额外推理延迟。 在生产环境中部署时，我们可以显式地计算并存储 $W = W_0 + BA$ 并像往常一样进行推理。请注意， W_0 和 BA 都在 $\mathbb{R}^{d \times k}$ 中。当我们需要切换到另一个下游任务时，我们可以通过减去 BA 然后加上不同的 $B'A'$ 来恢复 W_0 ，这是一个快速操作，内存开销非常小。关键在于，这

²它们与权重相比，仅代表极少数参数。³这是适应困难任务的必然结果。

guarantees that we do not introduce any additional latency during inference compared to a fine-tuned model by construction.

4.2 APPLYING LoRA TO TRANSFORMER

In principle, we can apply LoRA to any subset of weight matrices in a neural network to reduce the number of trainable parameters. In the Transformer architecture, there are four weight matrices in the self-attention module (W_q, W_k, W_v, W_o) and two in the MLP module. We treat W_q (or W_k, W_v) as a single matrix of dimension $d_{model} \times d_{model}$, even though the output dimension is usually sliced into attention heads. We limit our study to **only adapting the attention weights** for downstream tasks and freeze the MLP modules (so they are not trained in downstream tasks) both for simplicity and parameter-efficiency. We further study the effect on adapting different types of attention weight matrices in a Transformer in Section 7.1. We leave the empirical investigation of adapting the MLP layers, LayerNorm layers, and biases to a future work.

Practical Benefits and Limitations. The most significant benefit comes from the reduction in memory and storage usage. For a large Transformer trained with Adam, we reduce that VRAM usage by up to 2/3 if $r \ll d_{model}$ as we do not need to store the optimizer states for the frozen parameters. On GPT-3 175B, we reduce the VRAM consumption during training from 1.2TB to 350GB. With $r = 4$ and only the query and value projection matrices being adapted, the checkpoint size is reduced by roughly $10,000\times$ (from 350GB to 35MB)⁴. This allows us to train with significantly fewer GPUs and avoid I/O bottlenecks. Another benefit is that we can switch between tasks while deployed at a much lower cost by only swapping the LoRA weights as opposed to all the parameters. This allows for the creation of many customized models that can be swapped in and out on the fly on machines that store the pre-trained weights in VRAM. We also observe a 25% speedup during training on GPT-3 175B compared to full fine-tuning⁵ as we do not need to calculate the gradient for the vast majority of the parameters.

LoRA also has its limitations. For example, it is not straightforward to batch inputs to different tasks with different A and B in a single forward pass, if one chooses to absorb A and B into W to eliminate additional inference latency. Though it is possible to not merge the weights and dynamically choose the LoRA modules to use for samples in a batch for scenarios where latency is not critical.

5 EMPIRICAL EXPERIMENTS

We evaluate the downstream task performance of LoRA on RoBERTa (Liu et al., 2019), DeBERTa (He et al., 2021), and GPT-2 (Radford et al., b), before scaling up to GPT-3 175B (Brown et al., 2020). Our experiments cover a wide range of tasks, from natural language understanding (NLU) to generation (NLG). Specifically, we evaluate on the GLUE (Wang et al., 2019) benchmark for RoBERTa and DeBERTa. We follow the setup of Li & Liang (2021) on GPT-2 for a direct comparison and add WikiSQL (Zhong et al., 2017) (NL to SQL queries) and SAMSum (Gliwa et al., 2019) (conversation summarization) for large-scale experiments on GPT-3. See Appendix C for more details on the datasets we use. We use NVIDIA Tesla V100 for all experiments.

5.1 BASELINES

To compare with other baselines broadly, we replicate the setups used by prior work and reuse their reported numbers whenever possible. This, however, means that some baselines might only appear in certain experiments.

Fine-Tuning (FT) is a common approach for adaptation. During fine-tuning, the model is initialized to the pre-trained weights and biases, and all model parameters undergo gradient updates. A simple variant is to update only some layers while freezing others. We include one such baseline reported in prior work (Li & Liang, 2021) on GPT-2, which adapts just the last two layers (**FT^{Top2}**).

⁴We still need the 350GB model during deployment; however, storing 100 adapted models only requires 350GB + 35MB * 100 $\approx$ 354GB as opposed to 100 * 350GB $\approx$ 35TB.

⁵For GPT-3 175B, the training throughput for full fine-tuning is 32.5 tokens/s per V100 GPU; with the same number of weight shards for model parallelism, the throughput is 43.1 tokens/s per V100 GPU for LoRA.

保证了我们在推理过程中与微调模型相比，不会引入任何额外的延迟。

4.2 将LoRA应用于Transformer

原则上，我们可以将LoRA应用于神经网络中任意子集的权重矩阵，以减少可训练参数的数量。在Transformer架构中，自注意力模块中有四个权重矩阵（ W_q 、 W_k 、 W_v 、 W_o ），而MLP模块中有两个。我们将 W_q （或 W_k 、 W_v ）视为一个维度为 $d_{model} \times d_{model}$ 的矩阵，尽管输出维度通常会被切分成注意力头。我们仅限于研究仅对下游任务进行注意力权重的微调，并冻结MLP模块（因此它们在下游任务中不会被训练），这既是为了简化，也是为了参数效率。我们在第7.1节进一步研究了在Transformer中微调不同类型的注意力权重矩阵的影响。我们将MLP层、LayerNorm层和偏差的微调经验性研究留待未来工作。

实际效益与局限性。 最大的效益来自于内存和存储使用的减少。对于使用Adam训练的大型Transformer，如果我们像我们现在这样不存储冻结参数的优化器状态，我们可以将VRAM使用量减少高达 2/3。在GPT-3 175B上，我们将训练期间的VRAM消耗从1.2TB减少到350GB。使用 $r = 4$ ，并且仅适配查询和值投影矩阵，检查点大小减少了大约10,000 $\times$ （从350GB减少到35MB）⁴。这使得我们能够使用更少的GPU进行训练，并避免I/O瓶颈。另一个好处是，我们通过仅交换LoRA权重而不是所有参数，在部署时可以有更低的成本在任务之间切换。这允许创建许多可定制模型，这些模型可以在存储预训练权重的机器上随时切换进出。我们还观察到，与全微调相比，在GPT-3 175B上的训练速度提高了25%⁵，因为我们不需要为绝大多数参数计算梯度。

LoRA 也有其局限性。例如，如果选择将 A 和 B 吸收进 W 以消除额外的推理延迟，那么将具有不同 A 和 B 的不同任务的输入批量处理到单个前向传递中并不直接。尽管在延迟不是关键的情况下，可以不合并权重，并动态选择用于批量中样本的 LoRA 模块。

5 实验验证

我们在 RoBERTa（刘等人，2019年）、DeBERTa（何等人，2021年）和 GPT-2（拉德福德等人，b）上评估了 LoRA 的下游任务性能，然后再扩展到 GPT-3 175B（布朗等人，2020年）。我们的实验涵盖了从自然语言理解（NLU）到生成（NLG）的各种任务。具体来说，我们在 RoBERTa 和 DeBERTa 上使用 GLUE（王等人，2019年）基准进行评估。我们遵循 Li & Liang（2021年）在 GPT-2 上的设置进行直接比较，并添加了 WikiSQL（钟等人，2017年）（NL 到 SQL 查询）和 SAMSum（格利瓦等人，2019年）（对话摘要）以在 GPT-3 上进行大规模实验。有关我们使用的数据集的更多详细信息，请参阅附录 C。我们使用 NVIDIA Tesla V100 进行所有实验。

5.1 基线

为了与其他基线进行广泛比较，我们复制了先前工作所使用的设置，并在可能的情况下重用其报告的数值。然而，这意味着某些基线可能仅在特定的实验中出现。

微调 (FT) 是一种常见的适应方法。在微调过程中，模型被初始化为预训练权重和偏差，并且所有模型参数都进行梯度更新。一个简单的变体是仅更新某些层，同时冻结其他层。我们包含了一个先前工作中报告的此类基线（李 & 梁，2021）在GPT-2上的结果，该基线仅适应最后两层（**FT^{Top2}**）。

⁴我们在部署期间仍然需要 350GB 的模型；然而，存储 100 个适配的模型只需要 350GB + 35MB * 100 $\approx$ 354GB，而不是 100 * 350GB $\approx$ 35TB。⁵对于 GPT-3 175B，全微调的训练吞吐量是每 V100 GPU 32.5 个标记/秒；使用相同数量的权重分片进行模型并行，LoRA 的吞吐量是每 V100 GPU 43.1 个标记/秒。

Model & Method	# Trainable Parameters	MNLI	SST-2	MRPC	CoLA	QNLI	QQP	RTE	STS-B	Avg.
RoB _{base} (FT)*	125.0M	87.6	94.8	90.2	63.6	92.8	91.9	78.7	91.2	86.4
RoB _{base} (BitFit)*	0.1M	84.7	93.7	92.7	62.0	91.8	84.0	81.5	90.8	85.2
RoB _{base} (Adpt ^D)*	0.3M	87.1 $\pm$.0	94.2 $\pm$.1	88.5 $\pm$ 1.1	60.8 $\pm$.4	93.1 $\pm$.1	90.2 $\pm$.0	71.5 $\pm$ 2.7	89.7 $\pm$.3	84.4
RoB _{base} (Adpt ^D)*	0.9M	87.3 $\pm$.1	94.7 $\pm$.3	88.4 $\pm$.1	62.6 $\pm$.9	93.0 $\pm$.2	90.6 $\pm$.0	75.9 $\pm$ 2.2	90.3 $\pm$.1	85.4
RoB _{base} (LoRA)	0.3M	87.5 $\pm$.3	95.1$\pm$.2	89.7 $\pm$.7	63.4 $\pm$ 1.2	93.3$\pm$.3	90.8 $\pm$.1	86.6$\pm$.7	91.5$\pm$.2	87.2
RoB _{large} (FT)*	355.0M	90.2	96.4	90.9	68.0	94.7	92.2	86.6	92.4	88.9
RoB _{large} (LoRA)	0.8M	90.6$\pm$.2	96.2 $\pm$.5	90.9$\pm$1.2	68.2$\pm$1.9	94.9$\pm$.3	91.6 $\pm$.1	87.4$\pm$2.5	92.6$\pm$.2	89.0
RoB _{large} (Adpt ^P)†	3.0M	90.2 $\pm$.3	96.1 $\pm$.3	90.2 $\pm$.7	68.3$\pm$1.0	94.8$\pm$.2	91.9$\pm$.1	83.8 $\pm$ 2.9	92.1 $\pm$.7	88.4
RoB _{large} (Adpt ^P)†	0.8M	90.5$\pm$.3	96.6$\pm$.2	89.7 $\pm$ 1.2	67.8 $\pm$ 2.5	94.8$\pm$.3	91.7 $\pm$.2	80.1 $\pm$ 2.9	91.9 $\pm$.4	87.9
RoB _{large} (Adpt ^H)†	6.0M	89.9 $\pm$.5	96.2 $\pm$.3	88.7 $\pm$ 2.9	66.5 $\pm$ 4.4	94.7 $\pm$.2	92.1 $\pm$.1	83.4 $\pm$ 1.1	91.0 $\pm$ 1.7	87.8
RoB _{large} (Adpt ^H)†	0.8M	90.3 $\pm$.3	96.3 $\pm$.5	87.7 $\pm$ 1.7	66.3 $\pm$ 2.0	94.7 $\pm$.2	91.5 $\pm$.1	72.9 $\pm$ 2.9	91.5 $\pm$.5	86.4
RoB _{large} (LoRA)†	0.8M	90.6$\pm$.2	96.2 $\pm$.5	90.2$\pm$1.0	68.2 $\pm$ 1.9	94.8$\pm$.3	91.6 $\pm$.2	85.2$\pm$1.1	92.3$\pm$.5	88.6
DeB _{XXL} (FT)*	1500.0M	91.8	97.2	92.0	72.0	96.0	92.7	93.9	92.9	91.1
DeB _{XXL} (LoRA)	4.7M	91.9$\pm$.2	96.9 $\pm$.2	92.6$\pm$.6	72.4$\pm$1.1	96.0$\pm$.1	92.9$\pm$.1	94.9$\pm$.4	93.0$\pm$.2	91.3

Table 2: RoBERTa_{base}, RoBERTa_{large}, and DeBERTa_{XXL} with different adaptation methods on the GLUE benchmark. We report the overall (matched and mismatched) accuracy for MNLI, Matthew’s correlation for CoLA, Pearson correlation for STS-B, and accuracy for other tasks. Higher is better for all metrics. * indicates numbers published in prior works. † indicates runs configured in a setup similar to Houlsby et al. (2019) for a fair comparison.

Bias-only or BitFit is a baseline where we only train the bias vectors while freezing everything else. Contemporarily, this baseline has also been studied by BitFit (Zaken et al., 2021).

Prefix-embedding tuning (PreEmbed) inserts special tokens among the input tokens. These special tokens have trainable word embeddings and are generally not in the model’s vocabulary. Where to place such tokens can have an impact on performance. We focus on “prefixing”, which prepends such tokens to the prompt, and “infixing”, which appends to the prompt; both are discussed in Li & Liang (2021). We use l_p (resp. l_i) denote the number of prefix (resp. infix) tokens. The number of trainable parameters is $|\Theta| = d_{model} \times (l_p + l_i)$.

Prefix-layer tuning (PreLayer) is an extension to prefix-embedding tuning. Instead of just learning the word embeddings (or equivalently, the activations after the embedding layer) for some special tokens, we learn the activations after every Transformer layer. The activations computed from previous layers are simply replaced by trainable ones. The resulting number of trainable parameters is $|\Theta| = L \times d_{model} \times (l_p + l_i)$, where L is the number of Transformer layers.

Adapter tuning as proposed in Houlsby et al. (2019) inserts adapter layers between the self-attention module (and the MLP module) and the subsequent residual connection. There are two fully connected layers with biases in an adapter layer with a nonlinearity in between. We call this original design **Adapter^H**. Recently, Lin et al. (2020) proposed a more efficient design with the adapter layer applied only after the MLP module and after a LayerNorm. We call it **Adapter^L**. This is very similar to another deign proposed in Pfeiffer et al. (2021), which we call **Adapter^P**. We also include another baseline call AdapterDrop (Rücklé et al., 2020) which drops some adapter layers for greater efficiency (**Adapter^D**). We cite numbers from prior works whenever possible to maximize the number of baselines we compare with; they are in rows with an asterisk (*) in the first column. In all cases, we have $|\Theta| = \hat{L}_{Adpt} \times (2 \times d_{model} \times r + r + d_{model}) + 2 \times \hat{L}_{LN} \times d_{model}$ where $\hat{L}_{Adpt}$ is the number of adapter layers and $\hat{L}_{LN}$ the number of trainable LayerNorms (e.g., in Adapter^L).

LoRA adds trainable pairs of rank decomposition matrices in parallel to existing weight matrices. As mentioned in Section 4.2, we only apply LoRA to W_q and W_v in most experiments for simplicity. The number of trainable parameters is determined by the rank r and the shape of the original weights: $|\Theta| = 2 \times \hat{L}_{LoRA} \times d_{model} \times r$, where $\hat{L}_{LoRA}$ is the number of weight matrices we apply LoRA to.

模型 & 方法	#可训练参数	MNLI	SST-2	MRPC	CoLA	QNLI	QQP	RTE	STS-B	平均值
RoB _{base} (FT)*	125.0M	87.6	94.8	90.2	63.6	92.8	91.9	78.7	91.2	86.4
RoB _{base} (BitFit)*	0.1M	84.7	93.7	92.7	62.0	91.8	84.0	81.5	90.8	85.2
RoB _{base} (Adpt ^D)*	0.3M	87.1 $\pm$.0	94.2 $\pm$.1	88.5 $\pm$ 1.1	60.8 $\pm$.4	93.1 $\pm$.1	90.2 $\pm$.0	71.5 $\pm$ 2.7	89.7 $\pm$.3	84.4
RoB _{base} (Adpt ^D)*	0.9M	87.3 $\pm$.1	94.7 $\pm$.3	88.4 $\pm$.1	62.6 $\pm$.9	93.0 $\pm$.2	90.6 $\pm$.0	75.9 $\pm$ 2.2	90.3 $\pm$.1	85.4
RoB _{base} (LoRA)	0.3M	87.5 $\pm$.3	95.1$\pm$.2	89.7 $\pm$.7	63.4 $\pm$ 1.2	93.3$\pm$.3	90.8 $\pm$.1	86.6$\pm$.7	91.5$\pm$.2	87.2
RoB _{large} (FT)*	355.0M	90.2	96.4	90.9	68.0	94.7	92.2	86.6	92.4	88.9
RoB _{large} (LoRA)	0.8M	90.6$\pm$.2	96.2 $\pm$.5	90.9$\pm$1.2	68.2$\pm$1.9	94.9$\pm$.3	91.6 $\pm$.1	87.4$\pm$2.5	92.6$\pm$.2	89.0
RoB _{large} (Adpt ^P)†	3.0M	90.2 $\pm$.3	96.1 $\pm$.3	90.2 $\pm$.7	68.3$\pm$1.0	94.8$\pm$.2	91.9$\pm$.1	83.8 $\pm$ 2.9	92.1 $\pm$.7	88.4
RoB _{large} (Adpt ^P)†	0.8M	90.5$\pm$.3	96.6$\pm$.2	89.7 $\pm$ 1.2	67.8 $\pm$ 2.5	94.8$\pm$.3	91.7 $\pm$.2	80.1 $\pm$ 2.9	91.9 $\pm$.4	87.9
RoB _{large} (Adpt ^H)†	6.0M	89.9 $\pm$.5	96.2 $\pm$.3	88.7 $\pm$ 2.9	66.5 $\pm$ 4.4	94.7 $\pm$.2	92.1 $\pm$.1	83.4 $\pm$ 1.1	91.0 $\pm$ 1.7	87.8
RoB _{large} (Adpt ^H)†	0.8M	90.3 $\pm$.3	96.3 $\pm$.5	87.7 $\pm$ 1.7	66.3 $\pm$ 2.0	94.7 $\pm$.2	91.5 $\pm$.1	72.9 $\pm$ 2.9	91.5 $\pm$.5	86.4
RoB _{large} (LoRA)†	0.8M	90.6$\pm$.2	96.2 $\pm$.5	90.2$\pm$1.0	68.2 $\pm$ 1.9	94.8$\pm$.3	91.6 $\pm$.2	85.2$\pm$1.1	92.3$\pm$.5	88.6
DeB _{XXL} (FT)*	1500.0M	91.8	97.2	92.0	72.0	96.0	92.7	93.9	92.9	91.1
DeB _{XXL} (LoRA)	4.7M	91.9$\pm$.2	96.9 $\pm$.2	92.6$\pm$.6	72.4$\pm$1.1	96.0$\pm$.1	92.9$\pm$.1	94.9$\pm$.4	93.0$\pm$.2	91.3

表2: RoBERTa_{base}、RoBERTa_{large}和DeBERTa_{XXL} 在GLUE基准上使用不同适应方法。我们报告了MNLI的整体（匹配和不匹配）准确率、CoLA的Matthew相关系数、STS-B的Pearson相关系数以及其他任务的准确率。所有指标越高越好。*表示先前工作中发表的数据。†表示在Houlsby等人（2019）的设置中配置的运行，以便进行公平比较。

仅偏差**BitFit** 是一个基线，我们仅训练偏差向量，同时冻结其他所有内容。目前，该基线也已被

BitFit（Zaken等人，2021）研究过。

前缀嵌入调优（PreEmbed） 在输入标记之间插入特殊标记。这些特殊标记具有可训练的词嵌入，并且通常不在模型的词汇表中。放置此类标记的位置可能会影响性能。我们关注“前缀嵌入”，即将此类标记添加到提示中，以及“中缀嵌入”，即将其附加到提示中；两者都在Li & Liang (2021)中讨论。我们使用 l_p (resp. l_i)表示前缀（resp. 中缀）标记的数量。可训练参数数量是 $|\Theta| = d_{model} \times (l_p + l_i)$ 。

前缀层微调（PreLayer）是前缀嵌入调优的扩展。它不仅学习一些特殊标记的词嵌入（或等价地，嵌入层之后的激活值），还学习每个Transformer层之后的激活值。之前层计算出的激活值被简单地替换为可训练的激活值。由此产生的可训练参数数量为

$|\Theta| = L \times d_{model} \times (l_p + l_i)$ ，其中 L 是Transformer层的数量。

适配器调整如Houlsby等人(2019)所提出，在自注意力模块（和MLP模块）与后续残差连接之间插入适配器层。一个适配器层包含两个带有偏差的全连接层，并在中间加入非线性。我们称这种原始设计为**Adapter^H**。最近，Lin等人(2020)提出了一种更高效的设计，适配器层仅应用于MLP模块之后以及LayerNorm之后。我们称其为**Adapter^L**。这与Pfeiffer等人(2021)提出的另一种设计非常相似，我们称之为**Adapter^P**。我们还包含另一个基线称为AdapterDrop (Rücklé et al., 2020)，它通过丢弃一些适配器层来提高效率(**Adapter^D**)。我们尽可能引用先前工作的编号，以最大化我们比较的基线数量；它们位于第一列带有星号(*)的行中。^^^在所有情况下，我们都有 $|\Theta| = L_{Adpt} \times (2 \times d_{model} \times r + r + d_{model}) + 2 \times \hat{L}_{LN} \times d_{model}$ 其中 L_{Adpt} 是适配器层的数量， $\hat{L}_{LN}$ 是可训练的LayerNorm的数量（例如，在Adapter^L中）。

LoRA 在现有的权重矩阵中并行添加可训练的秩分解矩阵对。如第4.2节所述，我们为简洁起见，在大多数实验中仅将LoRA应用于 W_q 和 W_v 。可训练参数的数量由秩 r 和原始权重的形状 $|\Theta| = 2 \times L_{LoRA} \times d_{model} \times r$ 决定，其中 L_{LoRA} 是我们应用LoRA的权重矩阵数量。

Model & Method	# Trainable Parameters	E2E NLG Challenge				
		BLEU	NIST	MET	ROUGE-L	CIDEr
GPT-2 M (FT)*	354.92M	68.2	8.62	46.2	71.0	2.47
GPT-2 M (Adapter ^L)*	0.37M	66.3	8.41	45.0	69.8	2.40
GPT-2 M (Adapter ^L)*	11.09M	68.9	8.71	46.1	71.3	2.47
GPT-2 M (Adapter ^H)	11.09M	67.3 $\pm$.6	8.50 $\pm$.07	46.0 $\pm$.2	70.7 $\pm$.2	2.44 $\pm$.01
GPT-2 M (FT ^{Top2})*	25.19M	68.1	8.59	46.0	70.8	2.41
GPT-2 M (PreLayer)*	0.35M	69.7	8.81	46.1	71.4	2.49
GPT-2 M (LoRA)	0.35M	70.4$\pm$.1	8.85$\pm$.02	46.8$\pm$.2	71.8$\pm$.1	2.53$\pm$.02
GPT-2 L (FT)*	774.03M	68.5	8.78	46.0	69.9	2.45
GPT-2 L (Adapter ^L)	0.88M	69.1 $\pm$.1	8.68 $\pm$.03	46.3 $\pm$.0	71.4 $\pm$.2	2.49$\pm$.0
GPT-2 L (Adapter ^L)	23.00M	68.9 $\pm$.3	8.70 $\pm$.04	46.1 $\pm$.1	71.3 $\pm$.2	2.45 $\pm$.02
GPT-2 L (PreLayer)*	0.77M	70.3	8.85	46.2	71.7	2.47
GPT-2 L (LoRA)	0.77M	70.4$\pm$.1	8.89$\pm$.02	46.8$\pm$.2	72.0$\pm$.2	2.47 $\pm$.02

Table 3: GPT-2 medium (M) and large (L) with different adaptation methods on the E2E NLG Challenge. For all metrics, higher is better. LoRA outperforms several baselines with comparable or fewer trainable parameters. Confidence intervals are shown for experiments we ran. * indicates numbers published in prior works.

5.2 ROBERTA BASE/LARGE

RoBERTa (Liu et al., 2019) optimized the pre-training recipe originally proposed in BERT (Devlin et al., 2019a) and boosted the latter’s task performance without introducing many more trainable parameters. While RoBERTa has been overtaken by much larger models on NLP leaderboards such as the GLUE benchmark (Wang et al., 2019) in recent years, it remains a competitive and popular pre-trained model for its size among practitioners. We take the pre-trained RoBERTa base (125M) and RoBERTa large (355M) from the HuggingFace Transformers library (Wolf et al., 2020) and evaluate the performance of different efficient adaptation approaches on tasks from the GLUE benchmark. We also replicate Houlsby et al. (2019) and Pfeiffer et al. (2021) according to their setup. To ensure a fair comparison, we make two crucial changes to how we evaluate LoRA when comparing with adapters. First, we use the same batch size for all tasks and use a sequence length of 128 to match the adapter baselines. Second, we initialize the model to the pre-trained model for MRPC, RTE, and STS-B, not a model already adapted to MNLI like the fine-tuning baseline. Runs following this more restricted setup from Houlsby et al. (2019) are labeled with †. The result is presented in Table 2 (Top Three Sections). See Section D.1 for details on the hyperparameters used.

5.3 DeBERTa XXL

DeBERTa (He et al., 2021) is a more recent variant of BERT that is trained on a much larger scale and performs very competitively on benchmarks such as GLUE (Wang et al., 2019) and SuperGLUE (Wang et al., 2020). We evaluate if LoRA can still match the performance of a fully fine-tuned DeBERTa XXL (1.5B) on GLUE. The result is presented in Table 2 (Bottom Section). See Section D.2 for details on the hyperparameters used.

5.4 GPT-2 MEDIUM/LARGE

Having shown that LoRA can be a competitive alternative to full fine-tuning on NLU, we hope to answer if LoRA still prevails on NLG models, such as GPT-2 medium and large (Radford et al., b). We keep our setup as close as possible to Li & Liang (2021) for a direct comparison. Due to space constraint, we only present our result on E2E NLG Challenge (Table 3) in this section. See Section F.1 for results on WebNLG (Gardent et al., 2017) and DART (Nan et al., 2020). We include a list of the hyperparameters used in Section D.3.

模型与方法	# 可训练参数	端到端自然语言生成挑战				
		BLEU	NIST	MET	ROUGE-L	CIDEr
GPT-2 M (微调)*	354.92M	68.2	8.62	46.2	71.0	2.47
GPT-2 M (Adapter ^L)*	0.37M	66.3	8.41	45.0	69.8	2.40
GPT-2 M (Adapter ^L)*	11.09M	68.9	8.71	46.1	71.3	2.47
GPT-2 M (Adapter ^H)	11.09M	67.3 $\pm$.6	8.50 $\pm$.07	46.0 $\pm$.2	70.7 $\pm$.2	2.44 $\pm$.01
GPT-2 M (FT ^{Top2})*	25.19M	68.1	8.59	46.0	70.8	2.41
GPT-2 M (预层)*	0.35M	69.7	8.81	46.1	71.4	2.49
GPT-2 M (LoRA)	0.35M	70.4$\pm$.1	8.85$\pm$.02	46.8$\pm$.2	71.8$\pm$.1	2.53$\pm$.02
GPT-2 L (FT)*	774.03M	68.5	8.78	46.0	69.9	2.45
GPT-2 L (Adapter ^L)	0.88M	69.1 $\pm$.1	8.68 $\pm$.03	46.3 $\pm$.0	71.4 $\pm$.2	2.49$\pm$.0
GPT-2 L (Adapter ^L)	23.00M	68.9 $\pm$.3	8.70 $\pm$.04	46.1 $\pm$.1	71.3 $\pm$.2	2.45 $\pm$.02
GPT-2 L (预层)*	0.77M	70.3	8.85	46.2	71.7	2.47
GPT-2 L (LoRA)	0.77M	70.4$\pm$.1	8.89$\pm$.02	46.8$\pm$.2	72.0$\pm$.2	2.47 $\pm$.02

表3: GPT-2中（M）和大（L）在端到端自然语言生成挑战中使用不同的适应方法。对于所有指标，越高越好。LoRA在可训练参数相当或更少的情况下优于几个基线。我们运行实验的置信区间已显示。*表示先前工作中发表的数据。

5.2 ROBERTA BASE/LARGE

RoBERTa (Liu等人，2019)优化了BERT (Devlin等人，2019a)最初提出的预训练配方，并在不引入许多更多可训练参数的情况下提升了后者的任务性能。近年来，虽然RoBERTa在GLUE基准（Wang等人，2019）等NLP排行榜上已被更大模型超越，但它仍然是一个在从业者中具有竞争力和受欢迎的预训练模型。我们从HuggingFace Transformers库（Wolf等人，2020）获取预训练的RoBERTa基础版（125M）和RoBERTa large（355M），并在GLUE基准的任务上评估不同高效适应方法的表现。我们还根据其设置复制了Houlsby等人（2019）和Pfeiffer等人（2021）的工作。为确保公平比较，在将LoRA与适配器进行比较时，我们对LoRA的评估方式进行了两项关键更改。首先，我们为所有任务使用相同的批大小，并使用128的序列长度以匹配适配器基线。其次，我们将模型初始化为MRPC、RTE和STS-B的预训练模型，而不是像微调基线那样已经适应了MNLI的模型。遵循Houlsby等人（2019）更严格设置的运行被标记为 †。结果展示在表2（前三节）。有关所使用的超参数的详细信息，请参见第D.1节。

5.3 DeBERTa XXL

DeBERTa (He等人，2021)是BERT的一个较新的变体，在更大规模上训练，并在GLUE（Wang等人，2019）和SuperGLUE（Wang等人，2020）等基准测试中表现出非常具有竞争力的性能。我们评估LoRA是否仍然能在GLUE上匹配完全微调的DeBERTa XXL（1.5B）的性能。结果在表2（底部部分）中呈现。有关使用的超参数的详细信息，请参见第D.2节。

5.4 GPT-2中

我们已经证明LoRA在NLU任务上可以作为全微调的竞争性替代方案，因此我们希望回答LoRA是否在NLG模型（如GPT-2中和大模型）上仍然占优（Radford等人，b）。为了进行直接比较，我们尽可能保持设置与Li & Liang（2021）一致。由于篇幅限制，本节仅展示我们在端到端自然语言生成挑战（表3）上的结果。有关WebNLG（Gardent等人，2017）和DART（Nan等人，2020）的结果，请参见第F.1节。我们在第D.3节列出了所使用的超参数列表。

Model&Method	# Trainable Parameters	WikiSQL	MNLI-m	SAMSum
		Acc. (%)	Acc. (%)	R1/R2/RL
GPT-3 (FT)	175,255.8M	73.8	89.5	52.0/28.0/44.5
GPT-3 (BitFit)	14.2M	71.3	91.0	51.3/27.4/43.5
GPT-3 (PreEmbed)	3.2M	63.1	88.6	48.3/24.2/40.5
GPT-3 (PreLayer)	20.2M	70.1	89.5	50.8/27.3/43.5
GPT-3 (Adapter ^H)	7.1M	71.9	89.8	53.0/28.9/44.8
GPT-3 (Adapter ^H)	40.1M	73.2	91.5	53.2/29.0/45.1
GPT-3 (LoRA)	4.7M	73.4	91.7	53.8/29.8/45.9
GPT-3 (LoRA)	37.7M	74.0	91.6	53.4/29.2/45.1

Table 4: Performance of different adaptation methods on GPT-3 175B. We report the logical form validation accuracy on WikiSQL, validation accuracy on MultiNLI-matched, and Rouge-1/2/L on SAMSum. LoRA performs better than prior approaches, including full fine-tuning. The results on WikiSQL have a fluctuation around $\pm 0.5\%$, MNLI-m around $\pm 0.1\%$, and SAMSum around $\pm 0.2/\pm 0.2/\pm 0.1$ for the three metrics.

5.5 SCALING UP TO GPT-3 175B

As a final stress test for LoRA, we scale up to GPT-3 with 175 billion parameters. Due to the high training cost, we only report the typical standard deviation for a given task over random seeds, as opposed to providing one for every entry. See Section D.4 for details on the hyperparameters used.

As shown in Table 4, LoRA matches or exceeds the fine-tuning baseline on all three datasets. Note that not all methods benefit monotonically from having more trainable parameters, as shown in Figure 2. We observe a significant performance drop when we use more than 256 special tokens for prefix-embedding tuning or more than 32 special tokens for prefix-layer tuning. This corroborates similar observations in Li & Liang (2021). While a thorough investigation into this phenomenon is out-of-scope for this work, we suspect that having more special tokens causes the input distribution to shift further away from the pre-training data distribution. Separately, we investigate the performance of different adaptation approaches in the low-data regime in Section F.3.

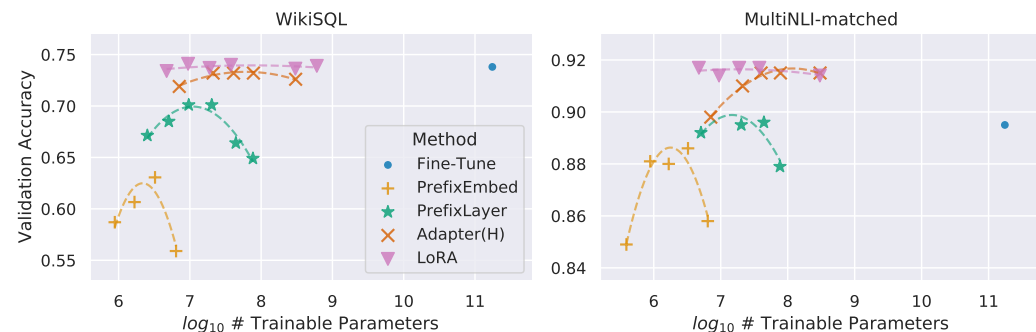


Figure 2: GPT-3 175B validation accuracy vs. number of trainable parameters of several adaptation methods on WikiSQL and MNLI-matched. LoRA exhibits better scalability and task performance. See Section F.2 for more details on the plotted data points.

6 RELATED WORKS

Transformer Language Models. Transformer (Vaswani et al., 2017) is a sequence-to-sequence architecture that makes heavy use of self-attention. Radford et al. (a) applied it to autoregressive language modeling by using a stack of Transformer decoders. Since then, Transformer-based language models have dominated NLP, achieving the state-of-the-art in many tasks. A new paradigm emerged with BERT (Devlin et al., 2019b) and GPT-2 (Radford et al., b) – both are large Transformer lan-

模型与方法	# 可训练参数	WikiSQL	MNLI-m	SAMSum
		准确率 (%)	准确率 (%)	R1/R2/RL
GPT-3 (FT)	175,255.8M	73.8	89.5	52.0/28.0/44.5
GPT-3 (BitFit)	14.2M	71.3	91.0	51.3/27.4/43.5
GPT-3 (PreEmbed)	3.2M	63.1	88.6	48.3/24.2/40.5
GPT-3 (PreLayer)	20.2M	70.1	89.5	50.8/27.3/43.5
GPT-3 (Adapter ^H)	7.1M	71.9	89.8	53.0/28.9/44.8
GPT-3 (Adapter ^H)	40.1M	73.2	91.5	53.2/29.0/45.1
GPT-3 (LoRA)	4.7M	73.4	91.7	53.8/29.8/45.9
GPT-3 (LoRA)	37.7M	74.0	91.6	53.4/29.2/45.1

表4: 不同适应方法在GPT-3 175B上的性能。我们报告了WikiSQL的逻辑形式验证准确率、MultiNLI匹配的验证准确率以及SAMSum的Rouge-1/2/L。LoRA的性能优于先前方法, 包括全微调。WikiSQL上的结果在 $\pm 0.5\%$ 左右波动, MNLI-m在 $\pm 0.1\%$ 左右, SAMSum在三个指标上分别为 $\pm 0.2/\pm 0.2/\pm 0.1$ 。

5.5 扩展至GPT-3 175B

作为LoRA的最终压力测试, 我们扩展到拥有1750亿参数的GPT-3。由于训练成本高昂, 我们仅报告给定任务在随机种子上的典型标准差, 而不是为每个条目提供一项。有关所使用超参数的详细信息, 请参见第D.4节。

如表4所示, LoRA在所有三个数据集上均达到或超过微调基线。请注意, 并非所有方法都能随着可训练参数的增加而单调受益, 如图2所示。当使用超过256个特殊标记进行前缀嵌入调优或超过32个特殊标记进行前缀层调优时, 我们观察到显著的性能下降。这证实了Li & Liang (2021)中的类似观察。尽管对这种现象的深入调查超出了本工作的范围, 但我们怀疑, 更多的特殊标记会导致输入分布进一步偏离预训练数据分布。此外, 我们在F.3节调查了低数据环境下不同适应方法的性能。

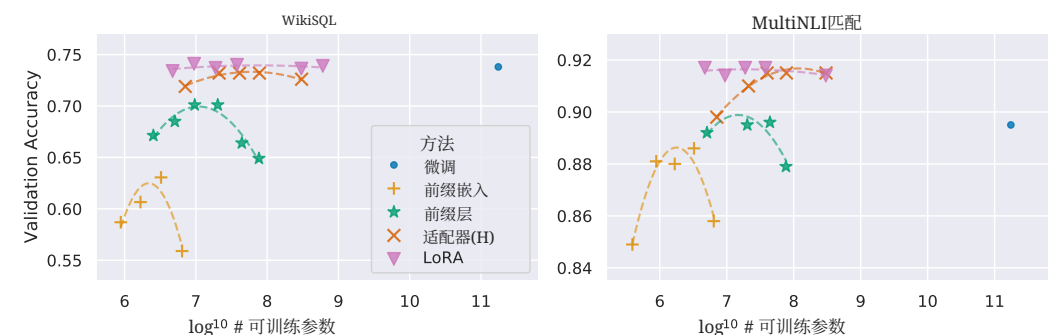


图2: GPT-3 175B验证准确率与WikiSQL和MNLI匹配上几种适应方法的可训练参数数量对比。LoRA表现出更好的可扩展性和任务性能。有关图中数据点的更多信息, 请参见第F.2节。

6 相关工作

Transformer语言模型。 Transformer (Vaswani等人, 2017年) 是一种序列到序列的架构, 大量使用了自注意力机制。Radford等人 (a) 通过使用堆叠的Transformer解码器将其应用于自回归语言建模。自那以后, 基于Transformer的语言模型主导了自然语言处理领域, 在许多任务中取得了最先进的成果。随着BERT (Devlin等人, 2019b) 和GPT-2 (Radford等人, b) 的出现, 一种新的范式应运而生——两者都是在大规模文本上训练的大型Transformer语言模型, 在通用领域数据上进行预训练后, 在特定任务数据上进行微调相比直接在特定任务数据上训练能提供显著的性能提升。训练更大规模的Transformer通常能带来更好的性能, 并仍然是活跃的研究方向。GPT-3 (布朗等人, 2020年) 是目前训练参数量最大的单个Transformer语言模型, 拥有175B个参数。

guage models trained on a large amount of text – where fine-tuning on task-specific data after pre-training on general domain data provides a significant performance gain compared to training on task-specific data directly. Training larger Transformers generally results in better performance and remains an active research direction. GPT-3 (Brown et al., 2020) is the largest single Transformer language model trained to-date with 175B parameters.

Prompt Engineering and Fine-Tuning. While GPT-3 175B can adapt its behavior with just a few additional training examples, the result depends heavily on the input prompt (Brown et al., 2020). This necessitates an empirical art of composing and formatting the prompt to maximize a model’s performance on a desired task, which is known as prompt engineering or prompt hacking. Fine-tuning retrains a model pre-trained on general domains to a specific task Devlin et al. (2019b); Radford et al. (a). Variants of it include learning just a subset of the parameters Devlin et al. (2019b); Collobert & Weston (2008), yet practitioners often retrain all of them to maximize the downstream performance. However, the enormity of GPT-3 175B makes it challenging to perform fine-tuning in the usual way due to the large checkpoint it produces and the high hardware barrier to entry since it has the same memory footprint as pre-training.

Parameter-Efficient Adaptation. Many have proposed inserting *adapter* layers between existing layers in a neural network (Houlsby et al., 2019; Rebuffi et al., 2017; Lin et al., 2020). Our method uses a similar bottleneck structure to impose a low-rank constraint on the weight updates. The key functional difference is that our learned weights can be merged with the main weights during inference, thus not introducing any latency, which is not the case for the adapter layers (Section 3). A comtemporary extension of adapter is COMPACTER (Mahabadi et al., 2021), which essentially parametrizes the adapter layers using Kronecker products with some predetermined weight sharing scheme. Similarly, combining LoRA with other tensor product-based methods could potentially improve its parameter efficiency, which we leave to future work. More recently, many proposed optimizing the input word embeddings in lieu of fine-tuning, akin to a continuous and differentiable generalization of prompt engineering (Li & Liang, 2021; Lester et al., 2021; Hambardzumyan et al., 2020; Liu et al., 2021). We include comparisons with Li & Liang (2021) in our experiment section. However, this line of works can only scale up by using more special tokens in the prompt, which take up available sequence length for task tokens when positional embeddings are learned.

Low-Rank Structures in Deep Learning. Low-rank structure is very common in machine learning. A lot of machine learning problems have certain intrinsic low-rank structure (Li et al., 2016; Cai et al., 2010; Li et al., 2018b; Grasedyck et al., 2013). Moreover, it is known that for many deep learning tasks, especially those with a heavily over-parametrized neural network, the learned neural network will enjoy low-rank properties after training (Oymak et al., 2019). Some prior works even explicitly impose the low-rank constraint when training the original neural network (Sainath et al., 2013; Povey et al., 2018; Zhang et al., 2014; Jaderberg et al., 2014; Zhao et al., 2016; Khodak et al., 2021; Denil et al., 2014); however, to the best of our knowledge, none of these works considers low-rank update to a frozen model for *adaptation to downstream tasks*. In theory literature, it is known that neural networks outperform other classical learning methods, including the corresponding (finite-width) neural tangent kernels (Allen-Zhu et al., 2019; Li & Liang, 2018) when the underlying concept class has certain low-rank structure (Ghorbani et al., 2020; Allen-Zhu & Li, 2019; Allen-Zhu & Li, 2020a). Another theoretical result in Allen-Zhu & Li (2020b) suggests that low-rank adaptations can be useful for adversarial training. In sum, we believe that our proposed low-rank adaptation update is well-motivated by the literature.

7 UNDERSTANDING THE LOW-RANK UPDATES

Given the empirical advantage of LoRA, we hope to further explain the properties of the low-rank adaptation learned from downstream tasks. Note that the low-rank structure not only lowers the hardware barrier to entry which allows us to run multiple experiments in parallel, but also gives better interpretability of how the update weights are correlated with the pre-trained weights. We focus our study on GPT-3 175B, where we achieved the largest reduction of trainable parameters (up to 10,000×) without adversely affecting task performances.

We perform a sequence of empirical studies to answer the following questions: 1) Given a parameter budget constraint, *which subset of weight matrices* in a pre-trained Transformer should we adapt

语言模型训练在大量文本上——在通用领域数据上进行预训练后，在特定任务数据上进行微调相比直接在特定任务数据上训练能提供显著的性能提升。训练更大规模的Transformer通常能带来更好的性能，并仍然是活跃的研究方向。GPT-3（布朗等人，2020年）是目前训练参数量最大的单个Transformer语言模型，拥有175B个参数。

提示工程与微调。虽然 GPT-3 175B 只需少量额外的训练示例就能调整其行为，但结果严重依赖于输入提示（布朗等人，2020）。这需要一种经验性的艺术来编写和格式化提示，以最大化模型在目标任务上的性能，这被称为提示工程或提示黑客。微调是对在通用领域预训练的模型进行特定任务的再训练（德夫林等人，2019b）；拉德福德等人（a）。它的变体包括仅学习参数的子集（德夫林等人，2019b）；科洛贝尔特与韦斯顿（2008），但从业者通常重新训练所有参数以最大化下游性能。然而，GPT-3 175B 的庞大规模使其难以以通常的方式执行微调，因为它产生的检查点很大，并且由于它具有与预训练相同的内存占用，硬件门槛也很高。

参数高效适应。许多研究者在神经网络现有层之间提出了插入适配器层（Houlsby等人，2019年；Rebuffi等人，2017年；Lin等人，2020年）。我们的方法使用类似的瓶颈结构对权重更新施加低秩约束。关键功能差异在于我们的学习权重可以在推理时与主权重合并，因此不会引入任何延迟，而适配器层则不会（第3节）。适配器的当代扩展是COMPACTER（Mahabadi等人，2021年），它本质上使用Kronecker积和预定的权重共享方案对适配器层进行参数化。类似地，将LoRA与其他基于张量积的方法结合可能会提高其参数效率，我们将其留待未来工作。最近，许多研究者在学习输入词嵌入而非微调方面提出了优化，类似于提示工程的连续和可微推广（李&梁，2021年；Lester等人，2021年；Hambardzumyan等人，2020年；刘等人，2021年）。我们在实验部分包含了与李&梁（2021年）的比较。然而，这条研究线只能通过提示中使用更多特殊标记来扩展，这些标记在位置嵌入学习时会占用可用于任务标记的序列长度。

深度学习中的低秩结构。低秩结构在机器学习中非常常见。许多机器学习问题具有内在的低秩结构（Li 等人，2016；Cai 等人，2010；Li 等人，2018b；Grasedyck 等人，2013）。此外，众所周知，对于许多深度学习任务，特别是那些具有高度过参数化神经网络的任务，学习到的神经网络在训练后将具有低秩特性（Oymak 等人，2019）。一些先前的工作甚至在训练原始神经网络时显式地施加低秩约束（Sainath 等人，2013；Povey 等人，2018；Zhang 等人，2014；Jaderberg 等人，2014；Zhao 等人，2016；Kho-dak 等人，2021；Denil 等人，2014）；然而，据我们所知，这些工作中的没有一项考虑对冻结模型进行低秩更新以适应下游任务。在理论文献中，众所周知，当底层概念类具有某些低秩结构时，神经网络在性能上优于其他经典学习方法，包括相应的（有限宽度）神经切线核（Allen-Zhu 等人，2019；Li & Liang，2018）（Ghorbani 等人，2020；Allen-Zhu & Li，2019；Allen-Zhu & Li，2020a）。Allen-Zhu & Li (2020b) 中的另一个理论结果表明，低秩适应可用于对抗训练。总之，我们认为我们提出的低秩适应更新得到了文献的有力支持。

7 理解低秩更新

鉴于LoRA的实证优势，我们希望进一步解释从下游任务中学习到的低秩适应的属性。请注意，低秩结构不仅降低了硬件门槛，使我们能够并行运行多个实验，还提供了更好的可解释性，说明更新权重如何与预训练权重相关。我们专注于GPT-3 175B，在那里我们实现了可训练参数的最大减少（高达10,000×），而没有对任务性能产生不利影响。

我们进行了一系列实证研究，以回答以下问题：1) 在参数预算约束下，预训练的Transformer中的哪些权重矩阵子集应该进行适应，以最大化下游性能？

to maximize downstream performance? 2) Is the “optimal” adaptation matrix ΔW *really rank-deficient*? If so, what is a good rank to use in practice? 3) What is the connection between ΔW and W ? Does ΔW highly correlate with W ? How large is ΔW comparing to W ?

We believe that our answers to question (2) and (3) shed light on the fundamental principles of using pre-trained language models for downstream tasks, which is a critical topic in NLP.

7.1 WHICH WEIGHT MATRICES IN TRANSFORMER SHOULD WE APPLY LoRA TO?

Given a limited parameter budget, which types of weights should we adapt with LoRA to obtain the best performance on downstream tasks? As mentioned in Section 4.2, we only consider weight matrices in the self-attention module. We set a parameter budget of 18M (roughly 35MB if stored in FP16) on GPT-3 175B, which corresponds to $r = 8$ if we adapt one type of attention weights or $r = 4$ if we adapt two types, for all 96 layers. The result is presented in Table 5.

	# of Trainable Parameters = 18M						
Weight Type Rank r	W_q 8	W_k 8	W_v 8	W_o 8	W_q, W_k 4	W_q, W_v 4	W_q, W_k, W_v, W_o 2
WikiSQL ($\pm 0.5\%$)	70.4	70.0	73.0	73.2	71.4	73.7	73.7
MultiNLI ($\pm 0.1\%$)	91.0	90.8	91.0	91.3	91.3	91.3	91.7

Table 5: Validation accuracy on WikiSQL and MultiNLI after applying LoRA to different types of attention weights in GPT-3, given the same number of trainable parameters. Adapting both W_q and W_v gives the best performance overall. We find the standard deviation across random seeds to be consistent for a given dataset, which we report in the first column.

Note that putting all the parameters in ΔW_q or ΔW_k results in significantly lower performance, while adapting both W_q and W_v yields the best result. This suggests that even a rank of four captures enough information in ΔW such that it is preferable to adapt more weight matrices than adapting a single type of weights with a larger rank.

7.2 WHAT IS THE OPTIMAL RANK r FOR LoRA?

We turn our attention to the effect of rank r on model performance. We adapt $\{W_q, W_v\}$, $\{W_q, W_k, W_v, W_o\}$, and just W_q for a comparison.

	Weight Type	$r = 1$	$r = 2$	$r = 4$	$r = 8$	$r = 64$
WikiSQL($\pm 0.5\%$)	W_q	68.8	69.6	70.5	70.4	70.0
	W_q, W_v	73.4	73.3	73.7	73.8	73.5
	W_q, W_k, W_v, W_o	74.1	73.7	74.0	74.0	73.9
MultiNLI ($\pm 0.1\%$)	W_q	90.7	90.9	91.1	90.7	90.7
	W_q, W_v	91.3	91.4	91.3	91.6	91.4
	W_q, W_k, W_v, W_o	91.2	91.7	91.7	91.5	91.4

Table 6: Validation accuracy on WikiSQL and MultiNLI with different rank r . To our surprise, a rank as small as one suffices for adapting both W_q and W_v on these datasets while training W_q alone needs a larger r . We conduct a similar experiment on GPT-2 in Section H.2.

Table 6 shows that, surprisingly, LoRA already performs competitively with a very small r (more so for $\{W_q, W_v\}$ than just W_q). This suggests the update matrix ΔW could have a very small “intrinsic rank”.⁶ To further support this finding, we check the overlap of the subspaces learned by different choices of r and by different random seeds. We argue that increasing r does not cover a more meaningful subspace, which suggests that a low-rank adaptation matrix is sufficient.

⁶However, we do not expect a small r to work for every task or dataset. Consider the following thought experiment: if the downstream task were in a different language than the one used for pre-training, retraining the entire model (similar to LoRA with $r = d_{model}$) could certainly outperform LoRA with a small r .

2) “最优”适配矩阵 ΔW 真的是秩亏的吗？如果是，实践中应该使用多大的秩？3) ΔW 和 W 之间有什么联系？ ΔW 与 W 高度相关吗？ ΔW 与 W 相比有多大？

我们相信，我们对问题（2）和（3）的回答揭示了使用预训练语言模型进行下游任务的基本原理，这是自然语言处理中的一个关键课题。

7.1 哪些权重矩阵在 Transformer 中应该应用 LoRA？

在有限的参数预算下，我们应该用 LoRA 调整哪些类型的权重以在下游任务中获得最佳性能？如第 4.2 节所述，我们仅考虑自注意力模块中的权重矩阵。我们在 GPT-3 175B 上设置了 18M 的参数预算（如果以 FP16 存储则约为 35MB），这对应于如果我们调整一种类型的注意力权重，则为 $r = 8$ ，或者如果我们调整两种类型的权重，则为 $r = 4$ ，对于所有 96 层。结果如表 5 所示。

	可训练参数数量 = 18M						
权重类型 秩 r	W_q 8	W_k 8	W_v 8	W_o 8	W_q, W_k 4	W_q, W_v 4	W_q, W_k, W_v, W_o 2
WikiSQL ($\pm 0.5\%$)	70.4	70.0	73.0	73.2	71.4	73.7	73.7
MultiNLI ($\pm 0.1\%$)	91.0	90.8	91.0	91.3	91.3	91.3	91.7

表 5：在 GPT-3 中对不同类型的注意力权重应用 LoRA 后，在 WikiSQL 和 MultiNLI 上的验证准确率，给定相同数量的可训练参数。调整 W_q 和 W_v 都能获得最佳整体性能。我们发现给定数据集的随机种子标准差是一致的，我们在第一列报告了该数据。

注意，将所有参数放入 ΔW_q 或 ΔW_k 会导致性能显著降低，而同时调整 W_q 和 W_v 会得到最佳结果。这表明即使秩为四也能在 ΔW 中捕获足够的信息，因此优先调整多个权重矩阵而不是用更大的秩调整单一类型的权重。

7.2 LoRA 的最优秩是多少？

我们将我们的注意力转向秩 r 对模型性能的影响。我们调整 $\{W_q, W_v, W_k, W_o\}$ ，以及仅 W_q 进行比较。

	权重类型	$r = 1$	$r = 2$	$r = 4$	$r = 8$	$r = 64$
WikiSQL($\pm 0.5\%$)	W_q	68.8	69.6	70.5	70.4	70.0
	W_q, W_v	73.4	73.3	73.7	73.8	73.5
	W_q, W_k, W_v, W_o	74.1	73.7	74.0	74.0	73.9
MultiNLI ($\pm 0.1\%$)	W_q	90.7	90.9	91.1	90.7	90.7
	W_q, W_v	91.3	91.4	91.3	91.6	91.4
	W_q, W_k, W_v, W_o	91.2	91.7	91.7	91.5	91.4

表6：不同秩 r 下 WikiSQL 和 MultiNLI 的验证准确率。令我们惊讶的是，仅一个较小的秩就足以在 W_q 和 W_v 上适配这些数据，而单独训练 W_q 则需要更大的 r 。我们在 H.2 节对 GPT-2 进行了类似的实验。

表6显示，出乎意料的是，LoRA 已经以非常小的 r （对 $\{W_q, W_v\}$ 而言比仅对 W_q 更明显）表现出具有竞争力的性能。这表明更新矩阵 ΔW 可能具有非常小的“固有秩”。⁶ 为了进一步支持这一发现，我们检查了由不同 r 选择和不同随机种子学习到的子空间的交集。我们认为增加 r 并不会覆盖更有意义的子空间，这表明低秩适配矩阵就足够了。

⁶然而，我们并不期望一个小的 r 能适用于所有任务或数据集。考虑以下思想实验：如果下游任务使用的语言与预训练时使用的语言不同，重新训练整个模型（类似于使用 $r = d_{model}$ 的 LoRA）当然会比使用小的 r 的 LoRA 表现更好。

Subspace similarity between different r . Given $A_{r=8}$ and $A_{r=64}$ which are the learned adaptation matrices with rank $r = 8$ and 64 using the *same pre-trained model*, we perform singular value decomposition and obtain the right-singular unitary matrices $U_{A_{r=8}}$ and $U_{A_{r=64}}$.⁷ We hope to answer: how much of the subspace spanned by the top i singular vectors in $U_{A_{r=8}}$ (for $1 \leq i \leq 8$) is contained in the subspace spanned by top j singular vectors of $U_{A_{r=64}}$ (for $1 \leq j \leq 64$)? We measure this quantity with a normalized subspace similarity based on the Grassmann distance (See Appendix G for a more formal discussion)

$$\phi(A_{r=8}, A_{r=64}, i, j) = \frac{\|U_{A_{r=8}}^{i\top} U_{A_{r=64}}^j\|_F^2}{\min(i, j)} \in [0, 1] \quad (4)$$

where $U_{A_{r=8}}^i$ represents the columns of $U_{A_{r=8}}$ corresponding to the top- i singular vectors.

$\phi(\cdot)$ has a range of $[0, 1]$, where 1 represents a complete overlap of subspaces and 0 a complete separation. See Figure 3 for how ϕ changes as we vary i and j . We only look at the 48th layer (out of 96) due to space constraint, but the conclusion holds for other layers as well, as shown in Section H.1.

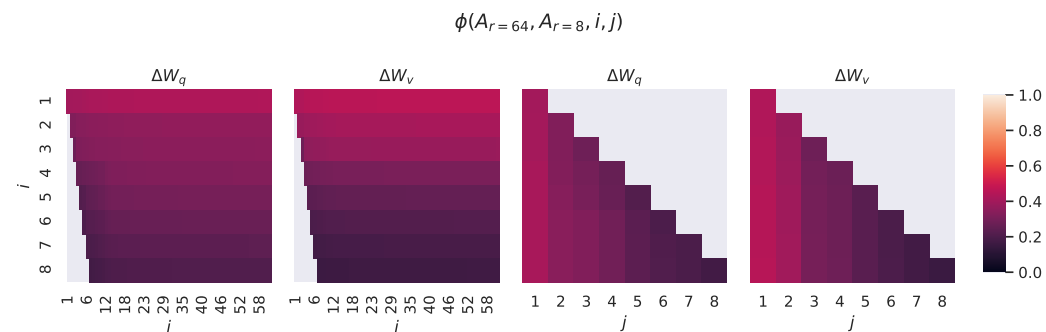


Figure 3: Subspace similarity between column vectors of $A_{r=8}$ and $A_{r=64}$ for both ΔW_q and ΔW_v . The third and the fourth figures zoom in on the lower-left triangle in the first two figures. The top directions in $r = 8$ are included in $r = 64$, and vice versa.

We make an *important observation* from Figure 3.

Directions corresponding to the top singular vector overlap significantly between $A_{r=8}$ and $A_{r=64}$, while others do not. Specifically, ΔW_v (resp. ΔW_q) of $A_{r=8}$ and ΔW_v (resp. ΔW_q) of $A_{r=64}$ share a subspace of dimension 1 with normalized similarity > 0.5 , providing an explanation of why $r = 1$ performs quite well in our downstream tasks for GPT-3.

Since both $A_{r=8}$ and $A_{r=64}$ are learned using the same pre-trained model, Figure 3 indicates that the top singular-vector directions of $A_{r=8}$ and $A_{r=64}$ are the most useful, while other directions potentially contain mostly random noises accumulated during training. Hence, the adaptation matrix can indeed have a very low rank.

Subspace similarity between different random seeds. We further confirm this by plotting the normalized subspace similarity between two randomly seeded runs with $r = 64$, shown in Figure 4. ΔW_q appears to have a higher “intrinsic rank” than ΔW_v , since more common singular value directions are learned by both runs for ΔW_q , which is in line with our empirical observation in Table 6. As a comparison, we also plot two random Gaussian matrices, which do not share any common singular value directions with each other.

7.3 HOW DOES THE ADAPTATION MATRIX ΔW COMPARE TO W ?

We further investigate the relationship between ΔW and W . In particular, does ΔW highly correlate with W ? (Or mathematically, is ΔW mostly contained in the top singular directions of W ?) Also,

⁷Note that a similar analysis can be carried out with B and the left-singular unitary matrices – we stick with A for our experiments.

不同之间的子空间相似性 r . 给定 $A_{r=8}$ 和 $A_{r=64}$ ，它们分别是使用相同预训练模型学习到的秩为 $r = 8$ 和 64 的适配矩阵，我们执行奇异值分解并得到右奇异酉矩阵 $U_{A_{r=8}}$ 和 $U_{A_{r=64}}$.⁷ 我们希望回答：由 $U_{A_{r=8}}$ （对于 $1 \leq i \leq 8$ ）中前 i 个奇异向量张成的子空间包含多少由 $U_{A_{r=64}}$ （对于 $1 \leq j \leq 64$ ）中前 j 个奇异向量张成的子空间？我们使用基于格拉姆距离的归一化子空间相似性来测量这个量（参见附录G以获得更正式的讨论）

$$\phi(A_{r=8}, A_{r=64}, i, j) = \frac{\|U_{A_{r=8}}^{i\top} U_{A_{r=64}}^j\|_F^2}{\min(i, j)} \in [0, 1] \quad (4)$$

$U_{A_{r=8}}^i$ 表示 $U_{A_{r=8}}$ 中对应于前 i 个奇异向量的列。

$\phi(\cdot)$ 的范围为 $[0, 1]$ ，其中 1 表示子空间完全重叠，0 表示完全分离。参见图3，了解 ϕ 如何随 i 和 j 的变化而变化。由于空间限制，我们仅考虑第48层（共96层），但结论对其他层同样适用，如H.1节所示。

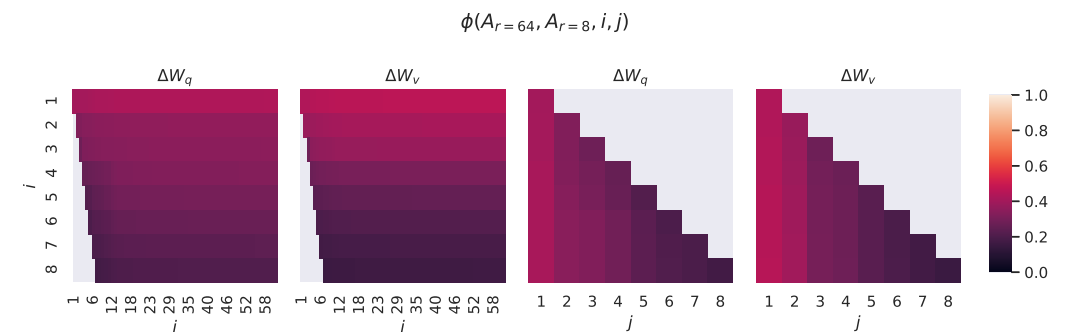


图3： $A_{r=8}$ 和 $A_{r=64}$ 的列向量之间的子空间相似性，分别针对 ΔW_q 和 ΔW_v 。第三和第四个图放大了前两个图左下三角的部分。 $r = 8$ 中的前向方向包含在 $r = 64$ 中，反之亦然。

我们从图3中得出一个重要观察。

对应于前奇异向量的方向在 $A_{r=8}$ 和 $A_{r=64}$ 之间显著重叠，而其他方向则不重叠。具体来说， $A_{r=8}$ 的 ΔW_v （resp. ΔW_q ）与 ΔW_v 的 ΔW_q （resp. $A_{r=64}$ ）共享一个维度为1的子空间，其归一化相似度为 > 0.5 ，这解释了为什么 $r = 1$ 在我们的GPT-3下游任务中表现相当好。

由于 $A_{r=8}$ 和 $A_{r=64}$ 都使用相同的预训练模型进行学习，图 3 表明 $A_{r=8}$ 和 $A_{r=64}$ 的前奇异向量方向是最有用的，而其他方向可能主要包含训练过程中积累的随机噪声。因此，适配矩阵确实可以具有非常低的秩。

不同随机种子的子空间相似性。 我们进一步通过绘制两个使用 $r = 64$ 的随机初始化运行之间的归一化子空间相似性来证实这一点，如图 4 所示。 ΔW_q 看起来比 ΔW_v 具有更高的“固有秩”，因为两个运行 ΔW_q 学习了更多的共同奇异值方向，这与我们在表 6 中的经验观察一致。作为比较，我们还绘制了两个随机高斯矩阵，它们彼此之间没有任何共同的奇异值方向。

7.3 适配矩阵 ΔW 与 W 如何比较？

我们进一步研究了 ΔW 与 W 之间的关系。特别是， ΔW 是否与 W 高度相关？（或者从数学上讲， ΔW 是否主要包含在 W 的前奇异方向中？）此外，

⁷注意，可以使用 B 和左奇异酉矩阵进行类似的分析——我们为实验坚持使用 A 。

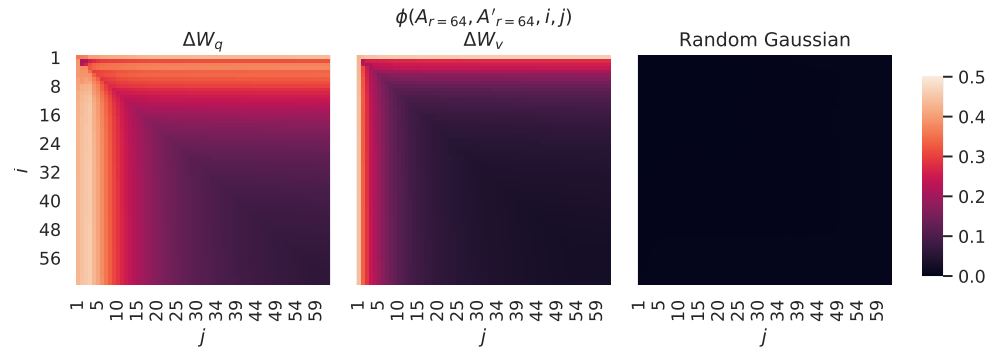


Figure 4: **Left and Middle:** Normalized subspace similarity between the column vectors of $A_{r=64}$ from two random seeds, for both ΔW_q and ΔW_v in the 48-th layer. **Right:** the same heat-map between the column vectors of two random Gaussian matrices. See Section H.1 for other layers.

how “large” is ΔW comparing to its corresponding directions in W ? This can shed light on the underlying mechanism for adapting pre-trained language models.

To answer these questions, we project W onto the r -dimensional subspace of ΔW by computing $U^\top W V^\top$, with U/V being the left/right singular-vector matrix of ΔW . Then, we compare the Frobenius norm between $\|U^\top W V^\top\|_F$ and $\|W\|_F$. As a comparison, we also compute $\|U^\top W V^\top\|_F$ by replacing U, V with the top r singular vectors of W or a random matrix.

	ΔW_q	$r=4$ W_q	Random	ΔW_q	$r=64$ W_q	Random
$\ U^\top W_q V^\top\ _F =$	0.32	21.67	0.02	1.90	37.71	0.33
$\ W_q\ _F = 61.95$	$\ \Delta W_q\ _F = 6.91$			$\ \Delta W_q\ _F = 3.57$		

Table 7: The Frobenius norm of $U^\top W_q V^\top$ where U and V are the left/right top r singular vector directions of either (1) ΔW_q , (2) W_q , or (3) a random matrix. The weight matrices are taken from the 48th layer of GPT-3.

We draw *several conclusions* from Table 7. First, ΔW has a stronger correlation with W compared to a random matrix, indicating that ΔW amplifies some features that are already in W . Second, instead of repeating the top singular directions of W , ΔW only *amplifies directions that are not emphasized in W* . Third, the amplification factor is rather huge: $21.5 \approx 6.91/0.32$ for $r=4$. See Section H.4 for why $r=64$ has a smaller amplification factor. We also provide a visualization in Section H.3 for how the correlation changes as we include more top singular directions from W_q . This suggests that the low-rank adaptation matrix potentially *amplifies the important features for specific downstream tasks that were learned but not emphasized in the general pre-training model*.

8 CONCLUSION AND FUTURE WORK

Fine-tuning enormous language models is prohibitively expensive in terms of the hardware required and the storage/switching cost for hosting independent instances for different tasks. We propose LoRA, an efficient adaptation strategy that neither introduces inference latency nor reduces input sequence length while retaining high model quality. Importantly, it allows for quick task-switching when deployed as a service by sharing the vast majority of the model parameters. While we focused on Transformer language models, the proposed principles are generally applicable to any neural networks with dense layers.

There are many directions for future works. 1) LoRA can be combined with other efficient adaptation methods, potentially providing orthogonal improvement. 2) The mechanism behind fine-tuning or LoRA is far from clear – how are features learned during pre-training transformed to do well on downstream tasks? We believe that LoRA makes it more tractable to answer this than full fine-

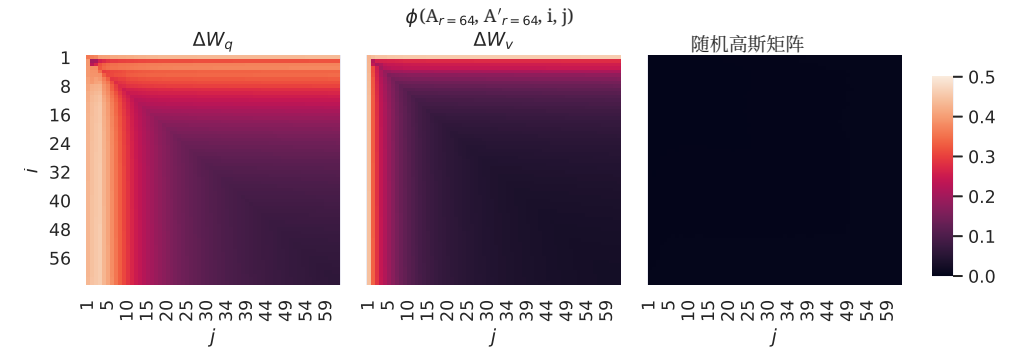


图4: **左和中间:** $A_{r=64}$ 的列向量从两个随机种子在48层中的归一化子空间相似度。对于 ΔW_q 和 ΔW_v 。 **右:** 两个随机高斯矩阵的列向量之间的相同热图。有关其他层的详细信息, 请参阅H.1节。

在 W 中, ΔW 的“大小”如何? 这可以揭示预训练语言模型自适应的潜在机制。

为了回答这些问题, 我们将 W 投影到 ΔW 的 r 维子空间上, 通过计算 $U^\top W V^\top$, 其中 U/V 是 ΔW 的左/右奇异向量矩阵。然后, 我们比较 $\|U^\top W V^\top\|_F$ 和 $\|W\|_F$ 之间的Frobenius范数。作为比较, 我们还通过用 W 或随机矩阵的 r 个最大奇异向量替换 U, V 来计算 $\|U^\top W V^\top\|_F$ 。

	ΔW_q	$r=4$ W_q	随机 ΔW	$r=64$ W_q	随机
$\ U^\top W_q V^\top\ _F =$	0.32	21.67	0.02	1.90	37.71
$\ W_q\ _F = 61.95$	$\ \Delta W_q\ _F = 6.91$			$\ \Delta W_q\ _F = 3.57$	

表7: $U^\top W_q V^\top$ 的Frobenius范数, 其中 U 和 V 是 (1) ΔW_q 、(2) W_q 或 (3) 随机矩阵的左/右顶部 r 奇异向量方向。权重矩阵取自GPT-3的第48层。

我们从表7中得出若干结论。首先, ΔW 与 W 的相关性比随机矩阵更强, 表明 ΔW 放大了 W 中已有的某些特征。其次, ΔW 没有重复 W 的顶部奇异方向, 而是仅放大在 W 中未被强调的方向。第三, 放大因子相当巨大: $21.5 \approx 6.91/0.32$ 对于 $r=4$ 。参见H.4节, 了解为何 $r=64$ 的放大因子较小。我们还提供了H.3节的可视化, 展示随着我们包含更多来自 W_q 的顶部奇异方向, 相关性如何变化。这表明低秩适配矩阵可能放大了特定下游任务的重要特征, 这些特征在通用预训练模型中已被学习但未被强调。

8 结论与未来工作

微调庞大的语言模型在硬件需求以及为不同任务托管独立实例所需的存储/切换成本方面是极其昂贵的。我们提出了LoRA, 这是一种高效的适应策略, 它既不会引入推理延迟, 也不会缩短输入序列长度, 同时又能保持高模型质量。重要的是, 当作为服务部署时, 它允许快速切换任务, 通过共享绝大多数模型参数来实现。虽然我们主要关注Transformer语言模型, 但所提出的原理适用于任何具有密集层的神经网络。

未来工作有许多方向。1) LoRA可以与其他高效适应方法结合, 可能提供正交改进。2) 微调或LoRA背后的机制远未明确——预训练期间学习的特征是如何转化为在下游任务中表现良好的? 我们认为LoRA比全量微调更容易回答这个问题。

tuning. 3) We mostly depend on heuristics to select the weight matrices to apply LoRA to. Are there more principled ways to do it? 4) Finally, the rank-deficiency of ΔW suggests that W could be rank-deficient as well, which can also be a source of inspiration for future works.

REFERENCES

Armen Aghajanyan, Luke Zettlemoyer, and Sonal Gupta. Intrinsic Dimensionality Explains the Effectiveness of Language Model Fine-Tuning. *arXiv:2012.13255 [cs]*, December 2020. URL <http://arxiv.org/abs/2012.13255>.

Zeyuan Allen-Zhu and Yuanzhi Li. What Can ResNet Learn Efficiently, Going Beyond Kernels? In *NeurIPS*, 2019. Full version available at <http://arxiv.org/abs/1905.10337>.

Zeyuan Allen-Zhu and Yuanzhi Li. Backward feature correction: How deep learning performs deep learning. *arXiv preprint arXiv:2001.04413*, 2020a.

Zeyuan Allen-Zhu and Yuanzhi Li. Feature purification: How adversarial training performs robust deep learning. *arXiv preprint arXiv:2005.10190*, 2020b.

Zeyuan Allen-Zhu, Yuanzhi Li, and Zhao Song. A convergence theory for deep learning via over-parameterization. In *ICML*, 2019. Full version available at <http://arxiv.org/abs/1811.03962>.

Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E. Hinton. Layer normalization, 2016.

Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language Models are Few-Shot Learners. *arXiv:2005.14165 [cs]*, July 2020. URL <http://arxiv.org/abs/2005.14165>.

Jian-Feng Cai, Emmanuel J Candès, and Zuowei Shen. A singular value thresholding algorithm for matrix completion. *SIAM Journal on optimization*, 20(4):1956–1982, 2010.

Daniel Cer, Mona Diab, Eneko Agirre, Inigo Lopez-Gazpio, and Lucia Specia. Semeval-2017 task 1: Semantic textual similarity multilingual and crosslingual focused evaluation. *Proceedings of the 11th International Workshop on Semantic Evaluation (SemEval-2017)*, 2017. doi: 10.18653/v1/s17-2001. URL <http://dx.doi.org/10.18653/v1/S17-2001>.

Ronan Collobert and Jason Weston. A unified architecture for natural language processing: deep neural networks with multitask learning. In *Proceedings of the 25th international conference on Machine learning*, ICML ’08, pp. 160–167, New York, NY, USA, July 2008. Association for Computing Machinery. ISBN 978-1-60558-205-4. doi: 10.1145/1390156.1390177. URL <https://doi.org/10.1145/1390156.1390177>.

Misha Denil, Babak Shakibi, Laurent Dinh, Marc’Aurelio Ranzato, and Nando de Freitas. Predicting parameters in deep learning, 2014.

Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding, 2019a.

Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. *arXiv:1810.04805 [cs]*, May 2019b. URL <http://arxiv.org/abs/1810.04805>. arXiv: 1810.04805.

William B. Dolan and Chris Brockett. Automatically constructing a corpus of sentential paraphrases. In *Proceedings of the Third International Workshop on Paraphrasing (IWP2005)*, 2005. URL <https://aclanthology.org/I05-5002>.

Claire Gardent, Anastasia Shimorina, Shashi Narayan, and Laura Perez-Beltrachini. The webnlg challenge: Generating text from rdf data. In *Proceedings of the 10th International Conference on Natural Language Generation*, pp. 124–133, 2017.

3) 我们主要依赖启发式方法来选择应用LoRA的权重矩阵。是否有更原则性的方法？4) 最后， ΔW 的秩亏表明 W 也可能秩亏，这也可以为未来工作提供灵感。

参考文献

Armen Aghajanyan、Luke Zettlemoyer 和 Sonal Gupta。内在维度解释了语言模型微调的有效性。*arXiv:2012.13255 [cs]*, 2020年12月。URL <http://arxiv.org/abs/2012.13255>。

Zeyuan Allen-Zhu 和 Yuanzhi Li。ResNet能高效学习什么，超越卷积核？收录于*NeurIPS*，2019年。完整版本可在 <http://arxiv.org/abs/1905.10337> 获取。

Zeyuan Allen-Zhu 和 Yuanzhi Li。反向特征校正：深度学习如何工作。收录于*arXiv* 预印本 *arXiv:2001.04413*，2020a。

Zeyuan Allen-Zhu 和 Yuanzhi Li。特征净化：对抗训练如何实现鲁棒深度学习。收录于*arXiv* 预印本 *arXiv:2005.10190*，2020b。

Zeyuan Allen-Zhu、Yuanzhi Li 和 Zhao Song。一种通过超参数化实现深度学习的收敛理论。发表于 *ICML*，2019。完整版本可在 <http://arxiv.org/abs/1811.03962> 获取。

Jimmy Lei Ba、Jamie Ryan Kiros 和 Geoffrey E. Hinton。层归一化，2016。

Tom B. Brown、Benjamin Mann、Nick Ryder、Melanie Subbiah、Jared Kaplan、Prafulla Dhariwal、Arvind Neelakantan、Pranav Shyam、Girish Sastry、Amanda Askell、Sandhini Agarwal、Ariel Herbert-Voss、Gretchen Krueger、Tom Henighan、Rewon Child、Aditya Ramesh、Daniel M. Ziegler、Jeffrey Wu、Clemens Winter、Christopher Hesse、Mark Chen、Eric Sigler、Mateusz Litwin、Scott Gray、Benjamin Chess、Jack Clark、Christopher Berner、Sam McCandlish、Alec Radford、Ilya Sutskever 和 Dario Amodei。语言模型是少样本学习器。*arXiv:2005.14165[cs]*, 2020年7月。URL <http://arxiv.org/abs/2005.14165>。

蔡建峰, 坎迪埃斯·埃马纽埃尔·J, 和沈祖伟. 矩阵补全的单值阈值算法. *SIAM 优化杂志*, 20(4):1956–1982, 2010.

Daniel Cer, Mona Diab, Eneko Agirre, Inigo Lopez-Gazpio, 和 Lucia Specia. Semeval-2017 任务 1: 语义文本相似度多语言和跨语言重点评估 . 第 11 届国际语义评估研讨会 (*SemEval-2017*) 会议录 , 2017. doi: 10.18653/v1/s17-2001. URL <http://dx.doi.org/10.18653/v1/S17-2001>.

Ronan Collobert 和 Jason Weston. 自然语言处理的统一架构：具有多任务学习的深度神经网络. 在 第 25 届国际机器学习会议会议录, ICML ’08, 第 160–167 页, 纽约, NY, USA, 2008 年 7 月. 计算机协会. ISBN 978-1-60558-205-4. doi: 10.1145/1390156.1390177. URL <https://doi.org/10.1145/1390156.1390177>.

MishaDenil、BabakShakibi、Laurent Dinh、Marc’Aurelio Ranzato以及NandodeFreitas。深度学习中预测参数，2014年。

Jacob Devlin、Ming-Wei Chang、Kenton Lee以及Kristina Toutanova。BERT：为语言理解而进行深度双向Transformer的预训练，2019a。

Jacob Devlin、Ming-Wei Chang、Kenton Lee以及Kristina Toutanova。BERT：为语言理解而进行深度双向Transformer的预训练。*arXiv:1810.04805 [cs]*, 2019b年5月。URL <http://arxiv.org/abs/1810.04805>. arXiv: 1810.04805。

William B. Dolan和Chris Brockett。自动构建一个句法释义语料库。在第三届释义国际研讨会论文集 (*IWP2005*) ，2005年。URL <https://aclanthology.org/I05-5002>。

克莱尔·加登特、阿纳斯塔西娅·希莫里纳、沙希·纳拉扬和劳拉·佩雷斯-贝特拉奇尼。WebNLG挑战：从RDF数据生成文本。在第十届自然语言生成国际会议论文集，第124-133页，2017年。

Behrooz Ghorbani, Song Mei, Theodor Misiakiewicz, and Andrea Montanari. When do neural networks outperform kernel methods? *arXiv preprint arXiv:2006.13409*, 2020.

Bogdan Gliwa, Iwona Mochol, Maciej Biesek, and Aleksander Wawer. Samsun corpus: A human-annotated dialogue dataset for abstractive summarization. *CoRR*, abs/1911.12237, 2019. URL <http://arxiv.org/abs/1911.12237>.

Lars Grasedyck, Daniel Kressner, and Christine Tobler. A literature survey of low-rank tensor approximation techniques. *GAMM-Mitteilungen*, 36(1):53–78, 2013.

Jihun Ham and Daniel D. Lee. Grassmann discriminant analysis: a unifying view on subspace-based learning. In *ICML*, pp. 376–383, 2008. URL <https://doi.org/10.1145/1390156.1390204>.

Karen Hambardzumyan, Hrant Khachatrian, and Jonathan May. WARP: Word-level Adversarial ReProgramming. *arXiv:2101.00121 [cs]*, December 2020. URL <http://arxiv.org/abs/2101.00121>. arXiv: 2101.00121.

Pengcheng He, Xiaodong Liu, Jianfeng Gao, and Weizhu Chen. Deberta: Decoding-enhanced bert with disentangled attention, 2021.

Neil Houlsby, Andrei Giurgiu, Stanislaw Jastrzebski, Bruna Morrone, Quentin de Laroussilhe, Andrea Gesmundo, Mona Attariyan, and Sylvain Gelly. Parameter-Efficient Transfer Learning for NLP. *arXiv:1902.00751 [cs, stat]*, June 2019. URL <http://arxiv.org/abs/1902.00751>.

Max Jaderberg, Andrea Vedaldi, and Andrew Zisserman. Speeding up convolutional neural networks with low rank expansions. *arXiv preprint arXiv:1405.3866*, 2014.

Mikhail Khodak, Neil Tenenholtz, Lester Mackey, and Nicolò Fusi. Initialization and regularization of factorized neural layers, 2021.

Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization, 2017.

Dmitry Lepikhin, HyoukJoong Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang, Maxim Krikun, Noam Shazeer, and Zhifeng Chen. Gshard: Scaling giant models with conditional computation and automatic sharding, 2020.

Brian Lester, Rami Al-Rfou, and Noah Constant. The Power of Scale for Parameter-Efficient Prompt Tuning. *arXiv:2104.08691 [cs]*, April 2021. URL <http://arxiv.org/abs/2104.08691>. arXiv: 2104.08691.

Chunyu Li, Heerad Farkhor, Rosanne Liu, and Jason Yosinski. Measuring the Intrinsic Dimension of Objective Landscapes. *arXiv:1804.08838 [cs, stat]*, April 2018a. URL <http://arxiv.org/abs/1804.08838>. arXiv: 1804.08838.

Xiang Lisa Li and Percy Liang. Prefix-Tuning: Optimizing Continuous Prompts for Generation. *arXiv:2101.00190 [cs]*, January 2021. URL <http://arxiv.org/abs/2101.00190>.

Yuanzhi Li and Yingyu Liang. Learning overparameterized neural networks via stochastic gradient descent on structured data. In *Advances in Neural Information Processing Systems*, 2018.

Yuanzhi Li, Yingyu Liang, and Andrej Risteski. Recovery guarantee of weighted low-rank approximation via alternating minimization. In *International Conference on Machine Learning*, pp. 2358–2367. PMLR, 2016.

Yuanzhi Li, Tengyu Ma, and Hongyang Zhang. Algorithmic regularization in over-parameterized matrix sensing and neural networks with quadratic activations. In *Conference On Learning Theory*, pp. 2–47. PMLR, 2018b.

Zhaojiang Lin, Andrea Madotto, and Pascale Fung. Exploring versatile generative language model via parameter-efficient transfer learning. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pp. 441–459, Online, November 2020. Association for Computational Linguistics. doi: 10.18653/v1/2020.findings-emnlp.41. URL <https://aclanthology.org/2020.findings-emnlp.41>.

Behrooz Ghorbani, Song Mei, Theodor Misiakiewicz, and Andrea Montanari. 当神经网络优于核方法时? *arXivpreprint arXiv:2006.13409*, 2020. Bogdan Gliwa, Iwona Mochol, Maciej Biesek, and Aleksander Wawer. Samsun 语料库: 用于抽象摘要的人 annotated 对话数据集。 *CoRR*, abs/1911.12237, 2019. URL <http://arxiv.org/abs/1911.12237>. Lars Grasedyck, Daniel Kressner, and Christine Tobler. 低秩张量逼近技术的文献综述。 *GAMM-Mitteilungen*, 36(1):53–78, 2013. Jihun Ham and Daniel D. Lee. 格拉斯曼判别分析: 基于子空间学习的统一观点。 在 *ICML*, pp. 376–383, 2008. URL <https://doi.org/10.1145/1390156.1390204>. Karen Hambardzumyan, Hrant Khachatrian, and Jonathan May. WARP: 词级对抗性重新编程。 *arXiv:2101.00121 [cs]*, 2020年12月。 URL <http://arxiv.org/abs/2101.00121>. arXiv: 2101.00121. Pengcheng He, Xiaodong Liu, Jianfeng Gao, and Weizhu Chen. Deberta: 具有解耦注意力的解码增强 bert, 2021. Neil Houlsby, Andrei Giurgiu, Stanislaw Jastrzebski, Bruna Morrone, Quentin de Laroussilhe, Andrea Gesmundo, Mona Attariyan, and Sylvain Gelly. NLP 的参数高效迁移学习。 *arXiv:1902.00751 [cs, stat]*, 2019年6月。 URL <http://arxiv.org/abs/1902.00751>. Max Jaderberg, Andrea Vedaldi, and Andrew Zisserman. 使用低秩扩展加速卷积神经网络。 *arXiv preprint arXiv:1405.3866*, 2014. Mikhail Khodak, Neil Tenenholtz, Lester Mackey, and Nicolò Fusi. 因子化神经层的初始化和正则化, 2021. Diederik P. Kingma and Jimmy Ba. Adam: 一种随机优化方法, 2017. Dmitry Lepikhin, HyoukJoong Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang, Maxim Krikun, Noam Shazeer, and Zhifeng Chen. Gshard: 使用条件计算和自动分片扩展巨型模型, 2020. Brian Lester, Rami Al-Rfou, and Noah Constant. 参数高效提示调优的规模力量。 *arXiv:2104.08691 [cs]*, 2021年4月。 URL <http://arxiv.org/abs/2104.08691>. arXiv: 2104.08691. Chunyu Li, Heerad Farkhor, Rosanne Liu, and Jason Yosinski. 测量目标景观的固有维度。 *arXiv:1804.08838 [cs, stat]*, 2018a年4月。 URL <http://arxiv.org/abs/1804.08838>. arXiv: 1804.08838. Xiang Lisa Li and Percy Liang. 前缀调优: 优化生成用连续提示。 *arXiv:2101.00190 [cs]*, 2021年1月。 URL <http://arxiv.org/abs/2101.00190>. Yuanzhi Li and Yingyu Liang. 通过结构化数据的随机梯度下降学习过参数化神经网络。 在 神经信息处理系统进展, 2018. Yuanzhi Li, Yingyu Liang, and Andrej Risteski. 通过交替最小化进行加权低秩逼近的恢复保证。 在 机器学习国际会议, pp. 2358–2367. PMLR, 2016. Yuanzhi Li, Tengyu Ma, and Hongyang Zhang. 过参数化矩阵感知和具有二次激活的神经网络的算法正则化。 在 学习理论会议, pp. 2–47. PMLR, 2018b. Zhaojiang Lin, Andrea Madotto, and Pascale Fung. 通过参数高效迁移学习探索多功能生成式语言模型。 在 计算语言学协会发现: *EMNLP 2020*, pp. 441–459, 在线, 2020年11月。 计算语言学协会. doi: 10.18653/v1/2020.findings-emnlp.41. URL <https://aclanthology.org/2020.findings-emnlp.41>.

Xiao Liu, Yanan Zheng, Zhengxiao Du, Ming Ding, Yujie Qian, Zhilin Yang, and Jie Tang. GPT Understands, Too. *arXiv:2103.10385 [cs]*, March 2021. URL <http://arxiv.org/abs/2103.10385>. arXiv: 2103.10385.

Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. Roberta: A robustly optimized bert pretraining approach, 2019.

Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. *arXiv preprint arXiv:1711.05101*, 2017.

Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization, 2019.

Rabeeh Karimi Mahabadi, James Henderson, and Sebastian Ruder. Compacter: Efficient low-rank hypercomplex adapter layers, 2021.

Linyong Nan, Dragomir Radev, Rui Zhang, Amrit Rau, Abhinand Sivaprasad, Chiachun Hsieh, Xiangru Tang, Aadit Vyas, Neha Verma, Pranav Krishna, et al. Dart: Open-domain structured data record to text generation. *arXiv preprint arXiv:2007.02871*, 2020.

Jekaterina Novikova, Ondřej Dušek, and Verena Rieser. The e2e dataset: New challenges for end-to-end generation. *arXiv preprint arXiv:1706.09254*, 2017.

Samet Oymak, Zalan Fabian, Mingchen Li, and Mahdi Soltanolkotabi. Generalization guarantees for neural networks via harnessing the low-rank structure of the jacobian. *arXiv preprint arXiv:1906.05392*, 2019.

Jonas Pfeiffer, Aishwarya Kamath, Andreas Rücklé, Kyunghyun Cho, and Iryna Gurevych. Adapterfusion: Non-destructive task composition for transfer learning, 2021.

Daniel Povey, Gaofeng Cheng, Yiming Wang, Ke Li, Hainan Xu, Mahsa Yarmohammadi, and Sanjeev Khudanpur. Semi-orthogonal low-rank matrix factorization for deep neural networks. In *Interspeech*, pp. 3743–3747, 2018.

Alec Radford, Karthik Narasimhan, Tim Salimans, and Ilya Sutskever. Improving Language Understanding by Generative Pre-Training. pp. 12, a.

Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language Models are Unsupervised Multitask Learners. pp. 24, b.

Pranav Rajpurkar, Robin Jia, and Percy Liang. Know what you don’t know: Unanswerable questions for squad. *CoRR*, abs/1806.03822, 2018. URL <http://arxiv.org/abs/1806.03822>.

Sylvestre-Alvise Rebuffi, Hakan Bilen, and Andrea Vedaldi. Learning multiple visual domains with residual adapters. *arXiv:1705.08045 [cs, stat]*, November 2017. URL <http://arxiv.org/abs/1705.08045>. arXiv: 1705.08045.

Andreas Rücklé, Gregor Geigle, Max Glockner, Tilman Beck, Jonas Pfeiffer, Nils Reimers, and Iryna Gurevych. Adapterdrop: On the efficiency of adapters in transformers, 2020.

Tara N Sainath, Brian Kingsbury, Vikas Sindhvani, Ebru Arisoy, and Bhuvana Ramabhadran. Low-rank matrix factorization for deep neural network training with high-dimensional output targets. In *2013 IEEE international conference on acoustics, speech and signal processing*, pp. 6655–6659. IEEE, 2013.

Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-lm: Training multi-billion parameter language models using model parallelism, 2020.

Richard Socher, Alex Perelygin, Jean Wu, Jason Chuang, Christopher D. Manning, Andrew Ng, and Christopher Potts. Recursive deep models for semantic compositionality over a sentiment treebank. In *Proceedings of the 2013 Conference on Empirical Methods in Natural Language Processing*, pp. 1631–1642, Seattle, Washington, USA, October 2013. Association for Computational Linguistics. URL <https://aclanthology.org/D13-1170>.

刘晓, 郑延安, 杜正晓, 丁明, 钱宇杰, 杨志林, 以及唐杰。GPT 也能理解。 *arXiv:2103.10385 [cs]*, 2021年3月。URL <http://arxiv.org/abs/2103.10385>。arXiv: 2103.10385。刘银汉, 奥特·迈尔, 高文南, 杜景飞, 乔希·曼达尔, 陈丹琪, 奥默·利维, 刘迈克, 泽特莫耶·卢克, 以及斯托扬诺夫·韦塞林。Roberta: 一种鲁棒优化的 BERT 预训练方法, 2019年。洛施基洛夫·伊利亚和胡特·弗兰克。解耦权重衰减正则化。 *arXiv* 预印本 *arXiv:1711.05101*, 2017 年。洛施基洛夫·伊利亚和胡特·弗兰克。解耦权重衰减正则化, 2019年。卡拉比·马哈巴迪·拉比, 詹姆斯·亨德森, 以及鲁德·塞巴斯蒂安。Compacter: 高效的低秩超复适配层, 2021年。

兰林勇, 拉戈米尔·拉德夫, 张睿, 阿米特·劳, 阿比南德·西瓦普拉萨德, 赵超群, 唐向如, 艾迪特·维亚斯, 内哈·维尔马, 普拉纳夫·克里希纳, 等。DART: 开放域结构化数据记录到文本生成. *arXiv* 预印本 *arXiv:2007.02871*, 2020.

叶卡捷琳娜·诺维科娃, 奥德雷·杜塞克, 和 维尔纳·里斯尔。E2E 数据集: 面向端到端生成的新挑战. *arXiv* 预印本 *arXiv:1706.09254*, 2017. 萨梅特·奥亚马克, 扎兰·法比安, 李明晨, 和 马赫迪·索尔坦洛塔比。通过利用雅可比矩阵的低秩结构为神经网络提供泛化保证. *arXiv* 预印本 *arXiv:1906.05392*, 2019.

Jonas Pfeiffer、Aishwarya Kamath、Andreas Rücklé、Kyunghyun Cho 和 IrynaGurevych。适配器融合：迁移学习中的非破坏性任务组合，2021。

Daniel Povey、Gaofeng Cheng、Yiming Wang、Ke Li、Hainan Xu、Mahsa Yarmohammadi 和 San- jeev Khudanpur。深度神经网络的半正交低秩矩阵分解。在*Interspeech*，第3743–3747页，2018。

Alec Radford、Karthik Narasimhan、Tim Salimans 和 Ilya Sutskever。通过生成式预训练改进语言理解。第12页，a。

Alec Radford、Jeffrey Wu、Rewon Child、David Luan、Dario Amodei 和 Ilya Sutskever。语言模型是无监督的多任务学习者。第24页，b。

Pranav Rajpurkar、Robin Jia 和 Percy Liang。知道你所不知道的：为 squad 准备的无解问题。 *CoRR*, abs/1806.03822, 2018. URL <http://arxiv.org/abs/1806.03822>。

Sylvestre-Alvise Rebuffi、Hakan Bilen 和 Andrea Vedaldi。使用残差适配器学习多个视觉域。 *arXiv:1705.08045 [cs, stat]*, 2017年11月。URL <http://arxiv.org/abs/1705.08045>。

arXiv: 1705.08045。Andreas Rücklé、Gregor Geigle、Max Glockner、Tilman Beck、Jonas Pfeiffer、Nils Reimers 和 Iryna Gurevych。Adapterdrop：关于转换器中适配器的效率，2020年。Tara N Sainath、Brian Kingsbury、Vikas Sindhvani、Ebru Arisoy 和 Bhuvana Ramabhadran。用于具有高维输出目标的深度神经网络训练的低秩矩阵分解。在 *2013 IEEE* 国际声学、语音和信号处理会议，第 6655–6659 页。IEEE，2013年。Mohammad Shoeybi、Mostofa Patwary、Raul Puri、Patrick LeGresley、Jared Casper 和 Bryan Catanzaro。Megatron-lm：使用模型并行性训练数十亿参数的语言模型，2020年。Richard Socher、Alex Perelygin、Jean Wu、Jason Chuang、Christopher D. Manning、Andrew Ng 和 Christopher Potts。递归深度模型用于情感树

库上的语义组合性。在 *2013 年自然语言处理经验方法会议论文集*，第 1631–1642 页，美国华盛顿西雅图，2013年10月。计算语言学协会。URL <https://aclanthology.org/D13-1170>。

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Proceedings of the 31st International Conference on Neural Information Processing Systems*, pp. 6000–6010, 2017.

Alex Wang, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel R. Bowman. Glue: A multi-task benchmark and analysis platform for natural language understanding, 2019.

Alex Wang, Yada Pruksachatkun, Nikita Nangia, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel R. Bowman. Superglue: A stickier benchmark for general-purpose language understanding systems, 2020.

Alex Warstadt, Amanpreet Singh, and Samuel R Bowman. Neural network acceptability judgments. *arXiv preprint arXiv:1805.12471*, 2018.

Adina Williams, Nikita Nangia, and Samuel Bowman. A broad-coverage challenge corpus for sentence understanding through inference. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers)*, pp. 1112–1122, New Orleans, Louisiana, June 2018. Association for Computational Linguistics. doi: 10.18653/v1/N18-1101. URL <https://www.aclweb.org/anthology/N18-1101>.

Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, Mariama Drame, Quentin Lhoest, and Alexander M. Rush. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pp. 38–45, Online, October 2020. Association for Computational Linguistics. URL <https://www.aclweb.org/anthology/2020.emnlp-demos.6>.

Greg Yang and Edward J. Hu. Feature Learning in Infinite-Width Neural Networks. *arXiv:2011.14522 [cond-mat]*, May 2021. URL <http://arxiv.org/abs/2011.14522>. arXiv: 2011.14522.

Elad Ben Zaken, Shauli Ravfogel, and Yoav Goldberg. Bitfit: Simple parameter-efficient fine-tuning for transformer-based masked language-models, 2021.

Yu Zhang, Ekapol Chuangsuwanich, and James Glass. Extracting deep neural network bottleneck features using low-rank matrix factorization. In *2014 IEEE international conference on acoustics, speech and signal processing (ICASSP)*, pp. 185–189. IEEE, 2014.

Yong Zhao, Jinyu Li, and Yifan Gong. Low-rank plus diagonal adaptation for deep neural networks. In *2016 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pp. 5005–5009. IEEE, 2016.

Victor Zhong, Caiming Xiong, and Richard Socher. Seq2sql: Generating structured queries from natural language using reinforcement learning. *CoRR*, abs/1709.00103, 2017. URL <http://arxiv.org/abs/1709.00103>.

A LARGE LANGUAGE MODELS STILL NEED PARAMETER UPDATES

Few-shot learning, or prompt engineering, is very advantageous when we only have a handful of training samples. However, in practice, we can often afford to curate a few thousand or more training examples for performance-sensitive applications. As shown in Table 8, fine-tuning improves the model performance drastically compared to few-shot learning on datasets large and small. We take the GPT-3 few-shot result on RTE from the GPT-3 paper (Brown et al., 2020). For MNLI-matched, we use two demonstrations per class and six in-context examples in total.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser 和 Illia Polosukhin. Attention is all you need. 在 第31届国际神经信息处理系统会议论文集, 第 6000–6010 页, 2017年。

Alex Wang、Amanpreet Singh、Julian Michael、Felix Hill、Omer Levy 和 Samuel R. Bowman。Glue: 一个用于自然语言理解的跨任务基准和分析平台, 2019年。

Alex Wang、Yada Pruksachatkun、Nikita Nangia、Amanpreet Singh、Julian Michael、Felix Hill、Omer Levy 和 Samuel R. Bowman。SuperGLUE: 一个更严格的通用语言理解系统基准, 2020年。

Alex Warstadt、Amanpreet Singh 和 Samuel R. Bowman。神经网络可接受性判断。*arXiv* 预印本 *arXiv:1805.12471*, 2018年。

Adina Williams、Nikita Nangia 和 Samuel Bowman。一个广泛覆盖的推理句法理解挑战语料库。在 北美计算语言学协会第 2018 年会议论文集: 人机语言技术, 第一卷 (长论文), 第 1112–1122 页, 新奥尔良, 路易斯安那州, 2018 年 6 月。计算语言学协会。doi: 10.18653/v1/N18-1101。URL <https://www.aclweb.org/anthology/N18-1101>。

托马斯·沃尔夫, 吕西安·德布, 维克多·桑, 朱利安·肖蒙, 克莱芒·达朗格, 安东尼·莫伊, 皮埃尔里克·西斯塔克, 蒂姆·劳, R·米·卢夫, 莫根·芬托维奇, 乔·戴维斯, 山姆·什莱弗, 帕特里克·冯·普拉滕, 克莱拉·马, 亚辛·杰尔尼特, 朱利安·普吕, 徐灿文, 特文·勒斯科, 西尔万·古格, 玛丽亚玛·德朗姆, 昆汀·勒韦斯特, 以及亚历山大·M·拉斯。Transformer: 最先进的自然语言处理。在《2020年自然语言处理经验方法会议系统演示会论文集》中, 第38-45页, 在线, 2020年10月。计算语言学协会。URL <https://www.aclweb.org/anthology/2020.emnlp-demos.6>。

格雷格·杨和爱德华·J·胡。无限宽度神经网络的特征学习。*arXiv:2011.14522 [cond-mat]*, 2021年5月。URL <http://arxiv.org/abs/2011.14522>。arXiv: 2011.14522。

Elad Ben Zaken、ShauliRavfogel和Yoav Goldberg。BitFit: 基于Transformer的掩码语言模型的简单参数高效微调, 2021年。

张宇、Ekapol Chuangsuwanich和詹姆斯·格拉斯。使用低秩矩阵分解提取深度神经网络瓶颈特征。在《2014 IEEE国际声学、语音与信号处理会议 (ICASSP) 》, 第185-189页。IEEE, 2014年。

赵勇、李金玉和龚一帆。深度神经网络的低秩加对角线自适应。在《2016 IEEE国际声学、语音与信号处理会议 (ICASSP) 》, 第5005-5009页。IEEE, 2016年。

钟维克、向凯明和理查德·索彻。Seq2sql: 使用强化学习从自然语言生成结构化查询。*CoRR*, abs/1709.00103, 2017年。URL <http://arxiv.org/abs/1709.00103>。

大型语言模型仍然需要参数更新

少样本学习, 或提示工程, 当只有少量训练样本时非常有利。然而, 在实践中, 对于性能敏感应用, 我们通常可以整理数千个或更多的训练示例。如表8所示, 与少样本学习相比, 微调在大小不同的数据集上显著提升了模型性能。我们从GPT-3论文 (布朗等人, 2020) 中取用RTE的GPT-3少样本结果。对于MNLI匹配, 我们每个类别使用两个演示, 总共使用六个上下文示例。

Method	MNLI-m (Val. Acc./%)	RTE (Val. Acc./%)
GPT-3 Few-Shot	40.6	69.0
GPT-3 Fine-Tuned	89.5	85.4

Table 8: Fine-tuning significantly outperforms few-shot learning on GPT-3 (Brown et al., 2020).

B INFERENCE LATENCY INTRODUCED BY ADAPTER LAYERS

Adapter layers are external modules added to a pre-trained model in a *sequential* manner, whereas our proposal, LoRA, can be seen as external modules added in a parallel manner. Consequently, adapter layers must be computed in addition to the base model, inevitably introducing additional latency. While as pointed out in Rücklé et al. (2020), the latency introduced by adapter layers can be mitigated when the model batch size and/or sequence length is large enough to full utilize the hardware parallelism. We confirm their observation with a similar latency study on GPT-2 medium and point out that there are scenarios, notably online inference where the batch size is small, where the added latency can be significant.

We measure the latency of a single forward pass on an NVIDIA Quadro RTX8000 by averaging over 100 trials. We vary the input batch size, sequence length, and the adapter bottleneck dimension r . We test two adapter designs: the original one by Houlsby et al. (2019), which we call Adapter^H, and a recent, more efficient variant by Lin et al. (2020), which we call Adapter^L. See Section 5.1 for more details on the designs. We plot the slow-down in percentage compared to the no-adapter baseline in Figure 5.

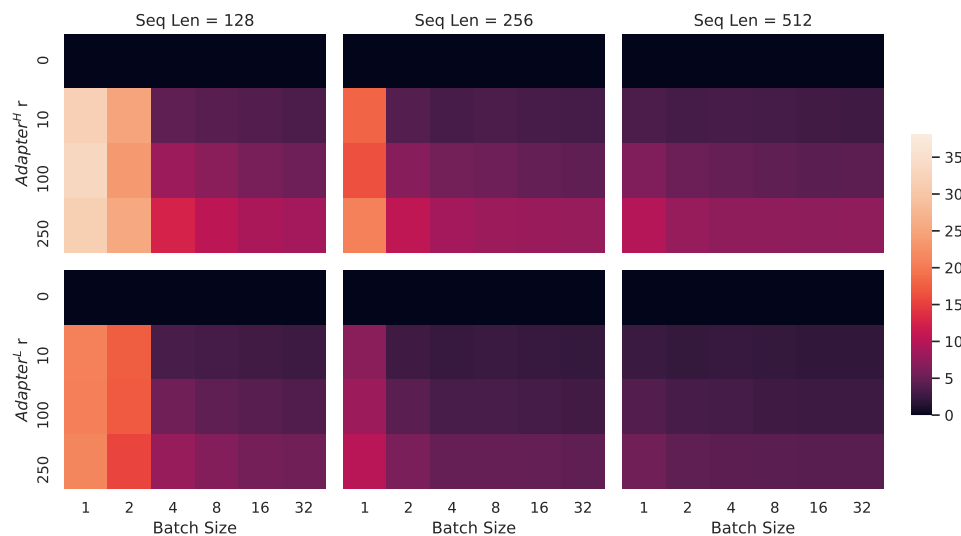


Figure 5: Percentage slow-down of inference latency compared to the no-adapter ($r = 0$) baseline. The top row shows the result for Adapter^H and the bottom row Adapter^L. Larger batch size and sequence length help to mitigate the latency, but the slow-down can be as high as over 30% in an online, short-sequence-length scenario. We tweak the colormap for better visibility.

C DATASET DETAILS

GLUE Benchmark is a wide-ranging collection of natural language understanding tasks. It includes MNLI (inference, Williams et al. (2018)), SST-2 (sentiment analysis, Socher et al. (2013)), MRPC (paraphrase detection, Dolan & Brockett (2005)), CoLA (linguistic acceptability, Warstadt et al. (2018)), QNLI (inference, Rajpurkar et al. (2018)), QQP⁸ (question-answering), RTE (inference),

⁸<https://quoradata.quora.com/First-Quora-Dataset-Release-Question-Pairs>

方法	MNLI-m (Val. Acc./%)	RTE (Val. Acc./%)
GPT-3 少样本	40.6	69.0
GPT-3 微调	89.5	85.4

表 8: 微调在 GPT-3 上显著优于少样本学习（布朗等人，2020）。

B 推理延迟由适配器层引入

适配器层是按<样式 id='1'>顺序</样式>添加到预训练模型的外部模块，而我们的提议 LoRA 可以看作是按并行方式添加的外部模块。因此，适配器层必须在基础模型之外进行计算，不可避免地会引入额外的延迟。虽然如 Rücklé 等人（2020）所指出的，当模型批大小和/或序列长度足够大以充分利用硬件并行性时，适配器层引入的延迟可以得到缓解。我们通过在 GPT-2 中进行的类似延迟研究证实了他们的观察，并指出在批量大小较小的情况下，尤其是在在线推理场景中，增加的延迟可能非常显著。

我们通过在 NVIDIA Quadro RTX8000 上对 100 次试验取平均来测量单次前向传递的延迟。我们改变输入批大小、序列长度和适配器瓶颈维度 r 。我们测试了两种适配器设计：Houlsby 等人（2019）提出的原始设计，我们称为 Adapter^H，以及 Lin 等人（2020）提出的一种更高效的变体，我们称为 Adapter^L。有关设计的更多细节，请参见第 5.1 节。我们在图 5 中绘制了与无适配器基线相比的百分比减速。

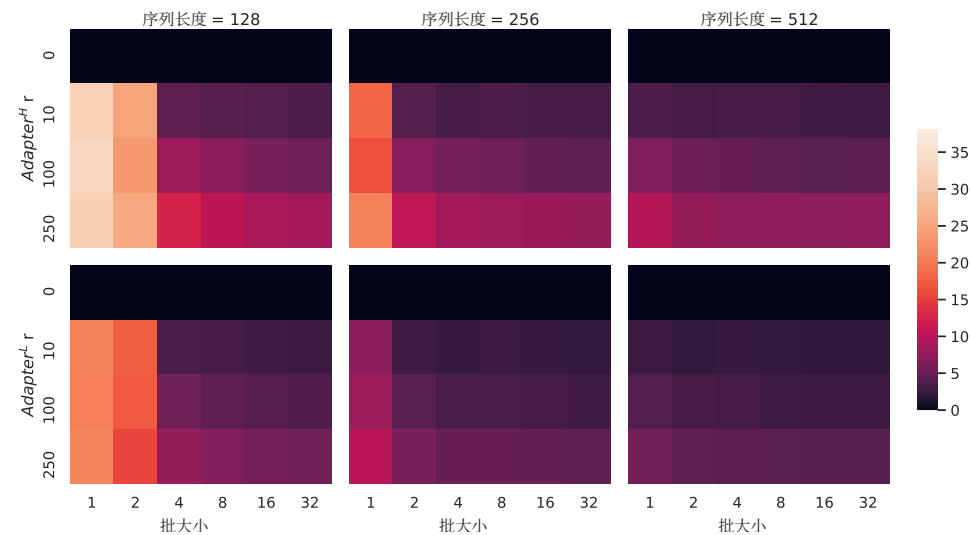


图 5: 与无适配器 ($r = 0$) 基线相比的推理延迟百分比减速。最上面一行显示了 Adapter^H 的结果，最下面一行显示了 Adapter^L 的结果。较大的批大小和序列长度有助于缓解延迟，但在在线、短序列长度的场景中，减速可能高达 30% 以上。我们调整了色彩映射以改善可见性。

C 数据集详情

GLUE 基准 是一个涵盖广泛的自然语言理解任务集合。它包括 MNLI（推理，Williams 等人（2018 年））、SST-2（情感分析，Socher 等人（2013 年））、MRPC（释义检测，Dolan & Brockett（2005 年））、CoLA（语言可接受性，Warstadt 等人（2018 年））、QNLI（推理，Rajpurkar 等人（2018 年））、QQP⁸（问答）、RTE（推理），

⁸<https://quoradata.quora.com/首个Quora数据集发布-问题对组>

and STS-B (textual similarity, Cer et al. (2017)). The broad coverage makes GLUE benchmark a standard metric to evaluate NLU models such as RoBERTa and DeBERTa. The individual datasets are released under different permissive licenses.

WikiSQL is introduced in Zhong et al. (2017) and contains 56,355/8,421 training/validation examples. The task is to generate SQL queries from natural language questions and table schemata. We encode context as $x = \{\text{table schema, query}\}$ and target as $y = \{\text{SQL}\}$. The dataset is release under the BSD 3-Clause License.

SAMSum is introduced in Gliwa et al. (2019) and contains 14,732/819 training/test examples. It consists of staged chat conversations between two people and corresponding abstractive summaries written by linguists. We encode context as ”\n” concatenated utterances followed by a ”\n\n”, and target as $y = \{\text{summary}\}$. The dataset is released under the non-commercial licence: Creative Commons BY-NC-ND 4.0.

E2E NLG Challenge was first introduced in Novikova et al. (2017) as a dataset for training end-to-end, data-driven natural language generation systems and is commonly used for data-to-text evaluation. The E2E dataset consists of roughly 42,000 training, 4,600 validation, and 4,600 test examples from the restaurant domain. Each source table used as input can have multiple references. Each sample input (x, y) consists of a sequence of slot-value pairs, along with a corresponding natural language reference text. The dataset is released under Creative Commons BY-NC-SA 4.0.

DART is an open-domain data-to-text dataset described in Nan et al. (2020). DART inputs are structured as sequences of ENTITY — RELATION — ENTITY triples. With 82K examples in total, DART is a significantly larger and more complex data-to-text task compared to E2E. The dataset is released under the MIT license.

WebNLG is another commonly used dataset for data-to-text evaluation (Gardent et al., 2017). With 22K examples in total WebNLG comprises 14 distinct categories, nine of which are seen during training. Since five of the 14 total categories are not seen during training, but are represented in the test set, evaluation is typically broken out by “seen” categories (S), “unseen” categories (U) and “all” (A). Each input example is represented by a sequence of SUBJECT — PROPERTY — OBJECT triples. The dataset is released under Creative Commons BY-NC-SA 4.0.

D HYPERPARAMETERS USED IN EXPERIMENTS

D.1 RoBERTa

We train using AdamW with a linear learning rate decay schedule. We sweep learning rate, number of training epochs, and batch size for LoRA. Following Liu et al. (2019), we initialize the LoRA modules to our best MNLI checkpoint when adapting to MRPC, RTE, and STS-B, instead of the usual initialization; the pre-trained model stays frozen for all tasks. We report the median over 5 random seeds; the result for each run is taken from the best epoch. For a fair comparison with the setup in Houlsby et al. (2019) and Pfeiffer et al. (2021), we restrict the model sequence length to 128 and used a fixed batch size for all tasks. Importantly, we start with the pre-trained RoBERTa large model when adapting to MRPC, RTE, and STS-B, instead of a model already adapted to MNLI. The runs with this restricted setup are marked with †. See the hyperparameters used in our runs in Table 9.

D.2 DeBERTa

We again train using AdamW with a linear learning rate decay schedule. Following He et al. (2021), we tune learning rate, dropout probability, warm-up steps, and batch size. We use the same model sequence length used by (He et al., 2021) to keep our comparison fair. Following He et al. (2021), we initialize the LoRA modules to our best MNLI checkpoint when adapting to MRPC, RTE, and STS-B, instead of the usual initialization; the pre-trained model stays frozen for all tasks. We report the median over 5 random seeds; the result for each run is taken from the best epoch. See the hyperparameters used in our runs in Table 10.

以及STS-B（文本相似度，Cer等人（2017年））。其广泛覆盖范围使GLUE基准成为评估RoBERTa和DeBERTa等自然语言理解模型的标准化指标。各个数据集以不同的宽松许可证发布。

WikiSQL 在Zhong等人(2017)中引入，包含56,355/8,421个训练/验证示例。该任务是从自然语言问题和表模式生成SQL查询。我们将上下文编码为 $x = \{\text{表模式,查询}\}$ ，将目标编码为 $y = \{\text{SQL}\}$ 。该数据集在BSD 3-Clause许可证下发布。

SAMSum 在 Gliwa 等人 (2019) 中被引入，并包含 14,732/819 个训练/测试示例。它由两个人之间的分阶段聊天对话以及相应的由语言学家撰写的抽象摘要组成。我们将上下文编码为”\n”连接的发言，后跟一个”\n\n”，将目标编码为 $y = \{\text{摘要}\}$ 。该数据集根据非商业许可发布：知识共享署名-非商业性使用-禁止演绎 4.0 许可。

端到端自然语言生成挑战首先在 Novikova 等人 (2017) 中被引入，作为一个用于训练端到端、数据驱动的自然语言生成系统的数据集，并且通常用于数据到文本评估。E2E 数据集包含大约 42,000 个训练样本、4,600 个验证样本和 4,600 个测试样本，来自餐厅领域。每个用作输入的源表可以具有多个引用。每个样本输入 (x, y) 由一个槽值对序列以及相应的自然语言参考文本组成。该数据集在 Creative Commons BY-NC-SA 4.0 下发布。

DART 是一个由Nan等人（2020）描述的开域数据到文本数据集。DART的输入被组织为实体—关系—实体三元组的序列。总共包含 82K 个示例，与E2E相比，DART是一个显著更大且更复杂的数据到文本任务。该数据集在MIT许可证下发布。

WebNLG 是另一个常用的数据集，用于数据到文本的评估（Gardent 等人，2017）。WebNLG 共有22K 个示例，包含14个不同的类别，其中训练过程中会看到其中的九个。由于14个总类别中有五个在训练过程中没有出现，但在测试集中有表示，因此评估通常按“已见”类别（S）、“未见”类别（U）和“全部”（A）进行划分。每个输入示例由一个SUBJECT — PROPERTY — OBJECT三元组序列表示。该数据集在Creative Commons BY-NC-SA 4.0下发布。

D 超参数用于实验

D.1 RoBERTa

我们使用 AdamW 并采用线性学习率衰减策略进行训练。我们为 LoRA 扫描学习率、训练轮数和批大小。遵循 Liu 等人 (2019) 的做法，在适应 MRPC、RTE 和 STS-B 时，我们将 LoRA 模块初始化为我们的最佳 MNLI 检查点，而不是常规初始化；预训练模型对所有任务都保持冻结。我们报告 5 个随机种子的中位数；每次运行的结果取自最佳轮次。为了与 Houlsby 等人 (2019) 和 Pfeiffer 等人 (2021) 的设置进行公平比较，我们将模型序列长度限制为 128，并对所有任务使用固定批大小。重要的是，在适应 MRPC、RTE 和 STS-B 时，我们使用预训练的 RoBERTa large 模型，而不是已经适应 MNLI 的模型。使用此限制设置的运行已标记为 †。请参见表 9 查看我们运行中使用的超参数。

D.2 DeBERTa

我们再次使用AdamW和线性学习率衰减计划进行训练。遵循He等人（2021年），我们调整学习率、Dropout概率、预热步数和批大小。我们使用与（He等人，2021年）相同的模型序列长度，以保持我们的比较公平。遵循He等人（2021年），在适应MRPC、RTE和STS-B时，我们将LoRA模块初始化到我们最佳的MNLI检查点，而不是通常的初始化；预训练模型对所有任务都保持冻结。我们报告5个随机种子的中位数；每次运行的结果来自最佳时期。请参见表10中我们运行时使用的超参数。

Method	Dataset	MNLI	SST-2	MRPC	CoLA	QNLI	QQP	RTE	STS-B
	Optimizer Warmup Ratio LR Schedule	AdamW 0.06 Linear							
RoBERTa base LoRA	Batch Size	16	16	16	32	32	16	32	16
	# Epochs	30	60	30	80	25	25	80	40
	Learning Rate	5E-04	5E-04	4E-04	4E-04	4E-04	5E-04	5E-04	4E-04
	LoRA Config.	$r_q = r_v = 8$							
	LoRA α Max Seq. Len.	8 512							
RoBERTa large LoRA	Batch Size	4	4	4	4	4	4	8	8
	# Epochs	10	10	20	20	10	20	20	30
	Learning Rate	3E-04	4E-04	3E-04	2E-04	2E-04	3E-04	4E-04	2E-04
	LoRA Config.	$r_q = r_v = 8$							
	LoRA α Max Seq. Len.	16 512							
RoBERTa large LoRA [†]	Batch Size	4							
	# Epochs	10	10	20	20	10	20	20	10
	Learning Rate	3E-04	4E-04	3E-04	2E-04	2E-04	3E-04	4E-04	2E-04
	LoRA Config.	$r_q = r_v = 8$							
	LoRA α Max Seq. Len.	16 128							
RoBERTa large Adpt ^P (3M) [†]	Batch Size	32							
	# Epochs	10	20	20	20	10	20	20	20
	Learning Rate	3E-05	3E-05	3E-04	3E-04	3E-04	3E-04	3E-04	3E-04
	Bottleneck r	64							
	Max Seq. Len.	128							
RoBERTa large Adpt ^P (0.8M) [†]	Batch Size	32							
	# Epochs	5	20	20	20	10	20	20	20
	Learning Rate	3E-04	3E-04	3E-04	3E-04	3E-04	3E-04	3E-04	3E-04
	Bottleneck r	16							
	Max Seq. Len.	128							
RoBERTa large Adpt ^H (6M) [†]	Batch Size	32							
	# Epochs	10	5	10	10	5	20	20	10
	Learning Rate	3E-05	3E-04	3E-04	3E-04	3E-04	3E-04	3E-04	3E-04
	Bottleneck r	64							
	Max Seq. Len.	128							
RoBERTa large Adpt ^H (0.8M) [†]	Batch Size	32							
	# Epochs	10	5	10	10	5	20	20	10
	Learning Rate	3E-04	3E-04	3E-04	3E-04	3E-04	3E-04	3E-04	3E-04
	Bottleneck r	8							
	Max Seq. Len.	128							

Table 9: The hyperparameters we used for RoBERTa on the GLUE benchmark.

D.3 GPT-2

We train all of our GPT-2 models using AdamW (Loshchilov & Hutter, 2017) with a linear learning rate schedule for 5 epochs. We use the batch size, learning rate, and beam search beam size described in Li & Liang (2021). Accordingly, we also tune the above hyperparameters for LoRA. We report the mean over 3 random seeds; the result for each run is taken from the best epoch. The hyperparameters used for LoRA in GPT-2 are listed in Table 11. For those used for other baselines, see Li & Liang (2021).

D.4 GPT-3

For all GPT-3 experiments, we train using AdamW (Loshchilov & Hutter, 2017) for 2 epochs with a batch size of 128 samples and a weight decay factor of 0.1. We use a sequence length of 384 for

方法	数据集	MNLI	SST-2	MRPC	CoLA	QNLI	QQP	RTE	STS-B
	优化器 预热比率 学习率调度	AdamW 0.06 线性							
RoBERTa 基础版 LoRA	批大小	16	16	16	32	32	16	32	16
	训练轮数	30	60	30	80	25	25	80	40
	学习率	5E-04	5E-04	4E-04	4E-04	4E-04	5E-04	5E-04	4E-04
	LoRA配置	$r_q = r_v = 8$							
	LoRA α 最大序列长度	8 512							
RoBERTa large LoRA	批大小	4	4	4	4	4	4	8	8
	训练轮数	10	10	20	20	10	20	20	30
	学习率	3E-04	4E-04	3E-04	2E-04	2E-04	3E-04	4E-04	2E-04
	LoRA配置	$r_q = r_v = 8$							
	LoRA α 最大序列长度	16 512							
RoBERTa large LoRA [†]	批大小	4							
	# 训练轮数	10	10	20	20	10	20	20	10
	学习率	3E-04	4E-04	3E-04	2E-04	2E-04	3E-04	4E-04	2E-04
	LoRA配置	$r_q = r_v = 8$							
	LoRA α 最大序列长度	16 128							
RoBERTa large Adpt ^P (3M) [†]	批大小	32							
	训练轮数	10	20	20	20	10	20	20	20
	学习率	3E-05	3E-05	3E-04	3E-04	3E-04	3E-04	3E-04	3E-04
	瓶颈 r	64							
	最大序列长度	128							
RoBERTa large Adpt ^P (0.8M) [†]	批大小	32							
	训练轮数	5	20	20	20	10	20	20	20
	学习率	3E-04	3E-04	3E-04	3E-04	3E-04	3E-04	3E-04	3E-04
	瓶颈 r	16							
	最大序列长度	128							
RoBERTa large Adpt ^H (6M) [†]	批大小	32							
	训练轮数	10	5	10	10	5	20	20	10
	学习率	3E-05	3E-04	3E-04	3E-04	3E-04	3E-04	3E-04	3E-04
	瓶颈 r	64							
	最大序列长度	128							
RoBERTa large Adpt ^H (0.8M) [†]	批大小	32							
	训练轮数	10	5	10	10	5	20	20	10
	学习率	3E-04	3E-04	3E-04	3E-04	3E-04	3E-04	3E-04	3E-04
	瓶颈 r	8							
	最大序列长度	128							

表9：我们在GLUE基准上为RoBERTa使用的超参数。

D.3 GPT-2

我们使用AdamW（Loshchilov & Hutter，2017）和线性学习率计划对所有的GPT-2模型进行5个时期的训练。我们使用Li & Liang（2021年）中描述的批大小、学习率和束搜索束大小。相应地，我们也为LoRA调整上述超参数。我们报告3个随机种子的平均值；每次运行的结果来自最佳时期。GPT-2中用于LoRA的超参数列于表11。其他基线使用的超参数，参见Li & Liang（2021年）。

D.4 GPT-3

对于所有 GPT-3 实验，我们使用 AdamW（Loshchilov & Hutter，2017）进行训练，训练 2 个 epoch，批大小为 128 个样本，权重衰减因子为 0.1。我们使用序列长度为 384。

Method	Dataset	MNLI	SST-2	MRPC	CoLA	QNLI	QQP	RTE	STS-B
	Optimizer					AdamW			
	Warmup Ratio					0.1			
	LR Schedule					Linear			
DeBERTa XXL LoRA	Batch Size	8	8	32	4	6	8	4	4
	# Epochs	5	16	30	10	8	11	11	10
	Learning Rate	1E-04	6E-05	2E-04	1E-04	1E-04	1E-04	2E-04	2E-04
	Weight Decay	0	0.01	0.01	0	0.01	0.01	0.01	0.1
	CLS Dropout	0.15	0	0	0.1	0.1	0.2	0.2	0.2
	LoRA Config.					$r_q = r_v = 8$			
	LoRA α					8			
	Max Seq. Len.	256	128	128	64	512	320	320	128

Table 10: The hyperparameters for DeBERTa XXL on tasks included in the GLUE benchmark.

Dataset	E2E	WebNLG	DART
	Training		
Optimizer	AdamW		
Weight Decay	0.01	0.01	0.0
Dropout Prob	0.1	0.1	0.0
Batch Size	8		
# Epoch	5		
Warmup Steps	500		
Learning Rate Schedule	Linear		
Label Smooth	0.1	0.1	0.0
Learning Rate	0.0002		
Adaptation	$r_q = r_v = 4$		
LoRA α	32		
	Inference		
Beam Size	10		
Length Penalty	0.9	0.8	0.8
no repeat ngram size	4		

Table 11: The hyperparameters for GPT-2 LoRA on E2E, WebNLG and DART.

WikiSQL (Zhong et al., 2017), 768 for MNLI (Williams et al., 2018), and 2048 for SAMSum (Gliwa et al., 2019). We tune learning rate for all method-dataset combinations. See Section D.4 for more details on the hyperparameters used. For prefix-embedding tuning, we find the optimal l_p and l_i to be 256 and 8, respectively, totalling $3.2M$ trainable parameters. We use $l_p = 8$ and $l_i = 8$ for prefix-layer tuning with $20.2M$ trainable parameters to obtain the overall best performance. We present two parameter budgets for LoRA: 4.7M ($r_q = r_v = 1$ or $r_v = 2$) and 37.7M ($r_q = r_v = 8$ or $r_q = r_k = r_v = r_o = 2$). We report the best validation performance from each run. The training hyperparameters used in our GPT-3 experiments are listed in Table 12.

E COMBINING LORA WITH PREFIX TUNING

LoRA can be naturally combined with existing prefix-based approaches. In this section, we evaluate two combinations of LoRA and variants of prefix-tuning on WikiSQL and MNLI.

LoRA+PrefixEmbed (LoRA+PE) combines LoRA with prefix-embedding tuning, where we insert $l_p + l_i$ special tokens whose embeddings are treated as trainable parameters. For more on prefix-embedding tuning, see Section 5.1.

LoRA+PrefixLayer (LoRA+PL) combines LoRA with prefix-layer tuning. We also insert $l_p + l_i$ special tokens; however, instead of letting the hidden representations of these tokens evolve natu-

方法	数据集	MNLI	SST-2	MRPC	CoLA	QNLI	QQP	RTE	STS-B
	优化器					AdamW			
	预热比率					0.1			
	学习率调度					线性			
DeBERTa XXL LoRA	批大小	8	8	32	4	6	8	4	4
	训练轮数	5	16	30	10	8	11	11	10
	学习率	1E-04	6E-05	2E-04	1E-04	1E-04	1E-04	2E-04	2E-04
	权重衰减	0	0.01	0.01	0	0.01	0.01	0.01	0.1
	分类头Dropout	0.15	0	0	0.1	0.1	0.2	0.2	0.2
	LoRA配置					$r_q = r_v = 8$			
	LoRA α					8			
	最大序列长度	256	128	128	64	512	320	320	128

表10：DeBERTa XXL在GLUE基准中包含的任务的超参数。

数据集	E2E	WebNLG	DART
	训练		
优化器	AdamW		
权重衰减	0.01	0.01	0.0
Dropout概率	0.1	0.1	0.0
批大小	8		
轮数	5		
预热步	500		
学习率计划	线性		
标签平滑	0.1	0.1	0.0
学习率	0.0002		
适应	$r_q = r_v = 4$		
LoRA α	32		
	推理		
束大小	10		
长度惩罚	0.9	0.8	0.8
不重复n元组大小	4		

表11：GPT-2 LoRA在E2E、WebNLG和DART上的超参数。

对于前缀嵌入调优，我们发现最优的 l_p 和 l_i 分别为 256 和 8，总共 $3.2M$ 个可训练参数。我们使用 $l_p = 8$ 和 $l_i = 8$ 进行前缀层微调，使用 $20.2M$ 个可训练参数以获得整体最佳性能。我们展示了 LoRA 的两个参数预算：4.7M ($r_q = r_v = 1$ 或 $r_v = 2$) 和 37.7M ($r_q = r_v = 8$ 或 $r_q = r_k = r_v = r_o = 2$)。我们报告了每次运行的最佳验证性能。我们在 GPT-3 实验中使用的训练超参数列于表 12 中。

E 结合 LoRA 与前缀调优

LoRA 可以与现有的基于前缀的方法自然结合。在本节中，我们评估了 LoRA 与前缀调优变体的两种组合在 WikiSQL 和 MNLI 上的表现。

LoRA+前缀嵌入 (LoRA+PE) 结合了 LoRA 与前缀嵌入调优，其中我们插入 $l_p + l_i$ 特殊标记，其嵌入被视为可训练参数。有关前缀嵌入调优的更多信息，请参见第 5.1 节。

LoRA+前缀层 (LoRA+PL) 结合了LoRA与前缀层微调。我们还插入了 $l_p + l_i$ 特殊标记；然而，我们没有让这些标记的隐藏表示自然演变

Hyperparameters	Fine-Tune	PreEmbed	PreLayer	BitFit	Adapter ^H	LoRA
Optimizer			AdamW			
Batch Size			128			
# Epoch			2			
Warmup Tokens			250,000			
LR Schedule			Linear			
Learning Rate	5.00E-06	5.00E-04	1.00E-04	1.6E-03	1.00E-04	2.00E-04

Table 12: The training hyperparameters used for different GPT-3 adaption methods. We use the same hyperparameters for all datasets after tuning learning rate.

rally, we replace them after every Transformer block with an input agnostic vector. Thus, both the embeddings and subsequent Transformer block activations are treated as trainable parameters. For more on prefix-layer tuning, see Section 5.1.

In Table 15, we show the evaluation results of LoRA+PE and LoRA+PL on WikiSQL and MultiNLI. First of all, LoRA+PE significantly outperforms both LoRA and prefix-embedding tuning on WikiSQL, which indicates that LoRA is somewhat orthogonal to prefix-embedding tuning. On MultiNLI, the combination of LoRA+PE doesn’t perform better than LoRA, possibly because LoRA on its own already achieves performance comparable to the human baseline. Secondly, we notice that LoRA+PL performs slightly worse than LoRA even with more trainable parameters. We attribute this to the fact that prefix-layer tuning is very sensitive to the choice of learning rate and thus makes the optimization of LoRA weights more difficult in LoRA+PL.

F ADDITIONAL EMPIRICAL EXPERIMENTS

F.1 ADDITIONAL EXPERIMENTS ON GPT-2

We also repeat our experiment on DART (Nan et al., 2020) and WebNLG (Gardent et al., 2017) following the setup of Li & Liang (2021). The result is shown in Table 13. Similar to our result on E2E NLG Challenge, reported in Section 5, LoRA performs better than or at least on-par with prefix-based approaches given the same number of trainable parameters.

Method	# Trainable Parameters	BLEU↑	DART MET↑	TER↓
GPT-2 Medium				
Fine-Tune	354M	46.2	0.39	0.46
Adapter ^L	0.37M	42.4	0.36	0.48
Adapter ^L	11M	45.2	0.38	0.46
FT ^{Top2}	24M	41.0	0.34	0.56
PrefLayer	0.35M	46.4	0.38	0.46
LoRA	0.35M	47.1 _{±.2}	0.39	0.46
GPT-2 Large				
Fine-Tune	774M	47.0	0.39	0.46
Adapter ^L	0.88M	45.7 _{±.1}	0.38	0.46
Adapter ^L	23M	47.1 _{±.1}	0.39	0.45
PrefLayer	0.77M	46.7	0.38	0.45
LoRA	0.77M	47.5 _{±.1}	0.39	0.45

Table 13: GPT-2 with different adaptation methods on DART. The variances of MET and TER are less than 0.01 for all adaption approaches.

超参数	微调	预嵌入	预层	BitFit	Adapter ^H	LoRA
优化器			AdamW			
批大小			128			
# 轮数			2			
预热令牌			250000			
学习率调度			线性			
学习率	5.00E-06	5.00E-04	1.00E-04	1.6E-03	1.00E-04	2.00E-04

表12：用于不同GPT-3适配方法的训练超参数。在调整学习率后，我们为所有数据集使用相同的超参数。

rally, 我们在每个Transformer模块后用与输入无关的向量替换它们。因此，嵌入和后续Transformer模块的激活都被视为可训练参数。有关前缀层微调的更多信息，请参见第5.1节。

在表15中，我们展示了LoRA+PE和LoRA+PL在WikiSQL和MultiNLI上的评估结果。首先，LoRA+PE在WikiSQL上显著优于LoRA和前缀嵌入调优，这表明LoRA与前缀嵌入调优在一定程度上是正交的。在MultiNLI上，LoRA+PE的组合表现并不优于LoRA，可能是因为LoRA本身已经达到了与人类基线相当的性能。其次，我们注意到LoRA+PL即使拥有更多可训练参数，表现也略差于LoRA。我们将此归因于前缀层微调对学习率的选择非常敏感，因此在LoRA+PL中使得LoRA权重的优化更加困难。

F 附加实验

F.1 GPT-2 附加实验

我们也重复了在 DART（Nan 等人，2020）和 WebNLG（Gardent 等人，2017）上的实验，遵循 Li & Liang（2021）的设置。结果如表 13 所示。与我们在第 5 节报告的 E2E NLG 挑战上的结果类似，LoRA 在可训练参数数量相同的情况下，表现优于或至少与基于前缀的方法相当。

方法	# 可训练参数	BLEU↑	DART MET↑	TER↓
GPT-2中				
微调	354M	46.2	0.39	0.46
适配器 ^L	0.37M	42.4	0.36	0.48
适配器 ^L	11M	45.2	0.38	0.46
FT ^{Top2}	24M	41.0	0.34	0.56
PrefLayer	0.35M	46.4	0.38	0.46
LoRA	0.35M	47.1 _{±.2}	0.39	0.46
GPT-2 大型				
微调	774M	47.0	0.39	0.46
适配器 ^L	0.88M	45.7 _{±.1}	0.38	0.46
适配器 ^L	23M	47.1 _{±.1}	0.39	0.45
PrefLayer	0.77M	46.7	0.38	0.45
LoRA	0.77M	47.5 _{±.1}	0.39	0.45

表13：在DART上使用不同适应方法的GPT-2。所有适应方法中，MET和TER的方差均小于。01。

Method	WebNLG								
	U	BLEU↑ S	A	U	MET↑ S	A	U	TER↓ S	A
GPT-2 Medium									
Fine-Tune (354M)	27.7	64.2	46.5	.30	.45	.38	.76	.33	.53
Adapter ^L (0.37M)	45.1	54.5	50.2	.36	.39	.38	.46	.40	.43
Adapter ^L (11M)	48.3	60.4	54.9	.38	.43	.41	.45	.35	.39
FT ^{Top2} (24M)	18.9	53.6	36.0	.23	.38	.31	.99	.49	.72
Prefix (0.35M)	45.6	62.9	55.1	.38	.44	.41	.49	.35	.40
LoRA (0.35M)	46.7 _{±.4}	62.1 _{±.2}	55.3_{±.2}	.38	.44	.41	.46	.33	.39
GPT-2 Large									
Fine-Tune (774M)	43.1	65.3	55.5	.38	.46	.42	.53	.33	.42
Adapter ^L (0.88M)	49.8_{±.0}	61.1 _{±.0}	56.0 _{±.0}	.38	.43	.41	.44	.35	.39
Adapter ^L (23M)	49.2 _{±.1}	64.7 _{±.2}	57.7_{±.1}	.39	.46	.43	.46	.33	.39
Prefix (0.77M)	47.7	63.4	56.3	.39	.45	.42	.48	.34	.40
LoRA (0.77M)	48.4 _{±.3}	64.0 _{±.3}	57.0 _{±.1}	.39	.45	.42	.45	.32	.38

Table 14: GPT-2 with different adaptation methods on WebNLG. The variances of MET and TER are less than 0.01 for all the experiments we ran. “U” indicates unseen categories, “S” indicates seen categories, and “A” indicates all categories in the test set of WebNLG.

F.2 ADDITIONAL EXPERIMENTS ON GPT-3

We present additional runs on GPT-3 with different adaptation methods in Table 15. The focus is on identifying the trade-off between performance and the number of trainable parameters.

F.3 LOW-DATA REGIME

To evaluate the performance of different adaptation approaches in the low-data regime. we randomly sample 100, 1k and 10k training examples from the full training set of MNLI to form the low-data MNLI- n tasks. In Table 16, we show the performance of different adaptation approaches on MNLI- n . To our surprise, PrefixEmbed and PrefixLayer performs very poorly on MNLI-100 dataset, with PrefixEmbed performing only slightly better than random chance (37.6% vs. 33.3%). PrefixLayer performs better than PrefixEmbed but is still significantly worse than Fine-Tune or LoRA on MNLI-100. The gap between prefix-based approaches and LoRA/Fine-tuning becomes smaller as we increase the number of training examples, which might suggest that prefix-based approaches are not suitable for low-data tasks in GPT-3. LoRA achieves better performance than fine-tuning on both MNLI-100 and MNLI-Full, and comparable results on MNLI-1k and MNLI-10K considering the (± 0.3) variance due to random seeds.

The training hyperparameters of different adaptation approaches on MNLI-n are reported in Table 17. We use a smaller learning rate for PrefixLayer on the MNLI-100 set, as the training loss does not decrease with a larger learning rate.

G MEASURING SIMILARITY BETWEEN SUBSPACES

In this paper we use the measure $\phi(A, B, i, j) = \psi(U_A^i, U_B^j) = \frac{\|U_A^{i\top} U_B\|_F^2}{\min_{\{i,j\}}}$ to measure the subspace similarity between two column orthonormal matrices $U_A^i \in \mathbb{R}^{d \times i}$ and $U_B^j \in \mathbb{R}^{d \times j}$, obtained by taking columns of the left singular matrices of A and B . We point out that this similarity is simply a reverse of the standard Projection Metric that measures distance between subspaces Ham & Lee (2008).

方法	WebNLG								
	U	BLEU↑ S	A	U	MET↑ S	A	U	TER↓ S	A
GPT-2中									
微调 (354M)	27.7	64.2	46.5	.30	.45	.38	.76	.33	.53
适配器 ^L (0.37M)	45.1	54.5	50.2	.36	.39	.38	.46	.40	.43
Adapter ^L (11M)	48.3	60.4	54.9	.38	.43	.41	.45	.35	.39
FT ^{Top2} (24M)	18.9	53.6	36.0	.23	.38	.31	.99	.49	.72
前缀 (0.35M)	45.6	62.9	55.1	.38	.44	.41	.49	.35	.40
LoRA (0.35M)	46.7 _{±.4}	62.1 _{±.2}	55.3_{±.2}	.38	.44	.41	.46	.33	.39
GPT-2 大型									
微调 (774M)	43.1	65.3	55.5	.38	.46	.42	.53	.33	.42
Adapter ^L (0.88M)	49.8_{±.0}	61.1 _{±.0}	56.0 _{±.0}	.38	.43	.41	.44	.35	.39
适配器 ^L (23M)	49.2 _{±.1}	64.7 _{±.2}	57.7_{±.1}	.39	.46	.43	.46	.33	.39
前缀 (0.77M)	47.7	63.4	56.3	.39	.45	.42	.48	.34	.40
LoRA (0.77M)	48.4 _{±.3}	64.0 _{±.3}	57.0 _{±.1}	.39	.45	.42	.45	.32	.38

表14: GPT-2在不同适应方法下在WebNLG上的表现。所有我们运行的所有实验中，MET和TER的方差均小于。01。“U”表示未见过的类别，“S”表示见过的类别，“A”表示WebNLG测试集中的所有类别。

F.2 GPT-3的附加实验

我们在表 15 中展示了使用不同适应方法在 GPT-3 上的额外运行结果。重点是识别性能和可训练参数数量之间的权衡。

F.3 低数据模式

为评估不同适应方法在低数据模式下的性能。我们从MNLI完整训练集中随机抽取100、1k和10k个训练示例，形成低数据MNLI- n 任务。在表16中，我们展示了不同适应方法在MNLI- n 上的性能。令我们惊讶的是，PrefixEmbed和PrefixLayer在MNLI-100数据集上表现非常差，PrefixEmbed仅略好于随机概率（37.6% vs. 33.3%）。PrefixLayer表现优于PrefixEmbed，但仍然显著劣于Fine-Tune或LoRA在MNLI-100上。随着训练示例数量的增加，基于前缀的方法与LoRA/Fine-tuning之间的差距缩小，这可能表明基于前缀的方法不适合GPT-3的低数据任务。LoRA在MNLI-100和MNLI-Full上均实现优于微调的性能，并在MNLI-1k和MNLI-10K上获得可比结果，考虑到(± 0.3)随机种子导致的方差。

MNLI-n 上不同适应方法训练的超参数在表 17 中报告。我们在 MNLI-100 集合上对前缀层使用较小的学习率，因为训练损失在更大的学习率下不会下降。

G 测量量子空间之间的相似性

在本文中，我们使用度量 $\phi(A, B, i, j) = \psi(U_A^i, U_B^j) = \frac{\|U_A^{i\top} U_B\|_{F_{\min\{i,j\}}}^2}{\|U_A^{i\top} U_B\|_{F_{\min\{i,j\}}}^2}$ 来衡量两个列正交矩阵 $U_A^i \in \mathbb{R}^{d \times i}$ 和 $U_B^j \in \mathbb{R}^{d \times j}$ 之间的子空间相似度，这两个矩阵是通过从 A 和 B 的左奇异矩阵中提取列得到的。我们指出，这种相似度仅仅是 Ham & Lee (2008) 中衡量量子空间之间距离的标准投影度量的反向操作。

Method	Hyperparameters	# Trainable Parameters	WikiSQL	MNLI-m
Fine-Tune	-	175B	73.8	89.5
PrefixEmbed	$l_p = 32, l_i = 8$	0.4 M	55.9	84.9
	$l_p = 64, l_i = 8$	0.9 M	58.7	88.1
	$l_p = 128, l_i = 8$	1.7 M	60.6	88.0
	$l_p = 256, l_i = 8$	3.2 M	63.1	88.6
	$l_p = 512, l_i = 8$	6.4 M	55.9	85.8
PrefixLayer	$l_p = 2, l_i = 2$	5.1 M	68.5	89.2
	$l_p = 8, l_i = 0$	10.1 M	69.8	88.2
	$l_p = 8, l_i = 8$	20.2 M	70.1	89.5
	$l_p = 32, l_i = 4$	44.1 M	66.4	89.6
	$l_p = 64, l_i = 0$	76.1 M	64.9	87.9
Adapter ^H	$r = 1$	7.1 M	71.9	89.8
	$r = 4$	21.2 M	73.2	91.0
	$r = 8$	40.1 M	73.2	91.5
	$r = 16$	77.9 M	73.2	91.5
	$r = 64$	304.4 M	72.6	91.5
LoRA	$r_v = 2$	4.7 M	73.4	91.7
	$r_q = r_v = 1$	4.7 M	73.4	91.3
	$r_q = r_v = 2$	9.4 M	73.3	91.4
	$r_q = r_k = r_v = r_o = 1$	9.4 M	74.1	91.2
	$r_q = r_v = 4$	18.8 M	73.7	91.3
	$r_q = r_k = r_v = r_o = 2$	18.8 M	73.7	91.7
	$r_q = r_v = 8$	37.7 M	73.8	91.6
	$r_q = r_k = r_v = r_o = 4$	37.7 M	74.0	91.7
	$r_q = r_v = 64$	301.9 M	73.6	91.4
	$r_q = r_k = r_v = r_o = 64$	603.8 M	73.9	91.4
LoRA+PE	$r_q = r_v = 8, l_p = 8, l_i = 4$	37.8 M	75.0	91.4
	$r_q = r_v = 32, l_p = 8, l_i = 4$	151.1 M	75.9	91.1
	$r_q = r_v = 64, l_p = 8, l_i = 4$	302.1 M	76.2	91.3
LoRA+PL	$r_q = r_v = 8, l_p = 8, l_i = 4$	52.8 M	72.9	90.2

Table 15: Hyperparameter analysis of different adaptation approaches on WikiSQL and MNLI. Both prefix-embedding tuning (PrefixEmbed) and prefix-layer tuning (PrefixLayer) perform worse as we increase the number of trainable parameters, while LoRA’s performance stabilizes. Performance is measured in validation accuracy.

Method	MNLI(m)-100	MNLI(m)-1k	MNLI(m)-10k	MNLI(m)-392K
GPT-3 (Fine-Tune)	60.2	85.8	88.9	89.5
GPT-3 (PrefixEmbed)	37.6	75.2	79.5	88.6
GPT-3 (PrefixLayer)	48.3	82.5	85.9	89.6
GPT-3 (LoRA)	63.8	85.6	89.2	91.7

Table 16: Validation accuracy of different methods on subsets of MNLI using GPT-3 175B. MNLI- n describes a subset with n training examples. We evaluate with the full validation set. LoRA performs exhibits favorable sample-efficiency compared to other methods, including fine-tuning.

To be concrete, let the singular values of $U_A^{i\top} U_B^j$ to be $\sigma_1, \sigma_2, \dots, \sigma_p$ where $p = \min\{i, j\}$. We know that the Projection Metric Ham & Lee (2008) is defined as:

$$d(U_A^i, U_B^j) = \sqrt{p - \sum_{i=1}^p \sigma_i^2} \in [0, \sqrt{p}]$$

方法	超参数	# 可训练参数	WikiSQL	MNLI-m
微调	-	175B	73.8	89.5
前缀嵌入	$l_p = 32, l_i = 8$	0.4M	55.9	84.9
	$l_p = 64, l_i = 8$	0.9M	58.7	88.1
	$l_p = 128, l_i = 8$	1.7 M	60.6	88.0
	$l_p = 256, l_i = 8$	3.2 M	63.1	88.6
	$l_p = 512, l_i = 8$	6.4 M	55.9	85.8
前缀层	$l_p = 2, l_i = 2$	5.1 M	68.5	89.2
	$l_p = 8, l_i = 0$	10.1 M	69.8	88.2
	$l_p = 8, l_i = 8$	20.2 M	70.1	89.5
	$l_p = 32, l_i = 4$	44.1 M	66.4	89.6
	$l_p = 64, l_i = 0$	76.1 M	64.9	87.9
Adapter ^H	$r = 1$	7.1 M	71.9	89.8
	$r = 4$	21.2 M	73.2	91.0
	$r = 8$	40.1 M	73.2	91.5
	$r = 16$	77.9 M	73.2	91.5
	$r = 64$	304.4 M	72.6	91.5
LoRA	$r_v = 2$	4.7 M	73.4	91.7
	$r_q = r_v = 1$	4.7 M	73.4	91.3
	$r_q = r_v = 2$	9.4 M	73.3	91.4
	$r_q = r_k = r_v = r_o = 1$	9.4 M	74.1	91.2
	$r_q = r_v = 4$	18.8 M	73.7	91.3
	$r_q = r_k = r_v = r_o = 2$	18.8 M	73.7	91.7
	$r_q = r_v = 8$	37.7 M	73.8	91.6
	$r_q = r_k = r_v = r_o = 4$	37.7 M	74.0	91.7
	$r_q = r_v = 64$	301.9 M	73.6	91.4
	$r_q = r_k = r_v = r_o = 64$	603.8 M	73.9	91.4
LoRA+PE	$r_q = r_v = 8, l_p = 8, l_i = 4$	37.8 M	75.0	91.4
	$r_q = r_v = 32, l_p = 8, l_i = 4$	151.1 M	75.9	91.1
	$r_q = r_v = 64, l_p = 8, l_i = 4$	302.1 M	76.2	91.3
LoRA+PL	$r_q = r_v = 8, l_p = 8, l_i = 4$	52.8 M	72.9	90.2

表 15：不同适应方法在 WikiSQL 和 MNLI 上的超参数分析。前缀嵌入调优（PrefixEmbed）和前缀层微调（PrefixLayer）的性能随着可训练参数数量的增加而变差，而 LoRA 的性能则保持稳定。性能以验证准确率衡量。

方法	MNLI(m)-100	MNLI(m)-1k	MNLI(m)-10k	MNLI(m)-392K
GPT-3（微调）	60.2	85.8	88.9	89.5
GPT-3（前缀嵌入调整）	37.6	75.2	79.5	88.6
GPT-3（前缀层）	48.3	82.5	85.9	89.6
GPT-3（LoRA）	63.8	85.6	89.2	91.7

表 16：使用 GPT-3 175B 在 MNLI 子集上不同方法的验证准确率。MNLI- n 描述了一个包含 n 个训练示例的子集。我们使用完整的验证集进行评估。与其它方法相比，LoRA 表现出有利的样本效率，包括微调。

具体来说，设 $U_A^{i\top} U_B^j$ 的奇异值为 $\sigma_1, \sigma_2, \dots, \sigma_p$ ，其中 $p = \leq \{i, j\}$ 。我们知道投影度量 Ham & Lee (2008) 定义为：

$$d(U_A^i, U_B^j) = \sqrt{p - \sum_{i=1}^p \sigma_i^2} \in [0, \sqrt{p}]$$

Hyperparameters	Adaptation	MNLI-100	MNLI-1k	MNLI-10K	MNLI-392K
Optimizer	-	AdamW			
Warmup Tokens	-	250,000			
LR Schedule	-	Linear			
Batch Size	-	20	20	100	128
# Epoch	-	40	40	4	2
Learning Rate	FineTune	5.00E-6			
	PrefixEmbed	2.00E-04	2.00E-04	4.00E-04	5.00E-04
	PrefixLayer LoRA	5.00E-05	5.00E-05	5.00E-05	1.00E-04
Adaptation-Specific	LoRA	2.00E-4			
	PrefixEmbed l_p	16	32	64	256
	PrefixEmbed l_i	8			
	PrefixTune	$l_p = l_i = 8$			
	LoRA	$r_q = r_v = 8$			

Table 17: The hyperparameters used for different GPT-3 adaptation methods on MNLI(m)- n .

where our similarity is defined as:

$$\phi(A, B, i, j) = \psi(U_A^i, U_B^j) = \frac{\sum_{i=1}^p \sigma_i^2}{p} = \frac{1}{p} \left(1 - d(U_A^i, U_B^j)^2 \right)$$

This similarity satisfies that if U_A^i and U_B^j share the same column span, then $\phi(A, B, i, j) = 1$. If they are completely orthogonal, then $\phi(A, B, i, j) = 0$. Otherwise, $\phi(A, B, i, j) \in (0, 1)$.

H ADDITIONAL EXPERIMENTS ON LOW-RANK MATRICES

We present additional results from our investigation into the low-rank update matrices.

H.1 CORRELATION BETWEEN LORA MODULES

See Figure 6 and Figure 7 for how the results presented in Figure 3 and Figure 4 generalize to other layers.

H.2 EFFECT OF r ON GPT-2

We repeat our experiment on the effect of r (Section 7.2) in GPT-2. Using the E2E NLG Challenge dataset as an example, we report the validation loss and test metrics achieved by different choices of r after training for 26,000 steps. We present our result in Table 18. The optimal rank for GPT-2 Medium is between 4 and 16 depending on the metric used, which is similar to that for GPT-3 175B. Note that the relationship between model size and the optimal rank for adaptation is still an open question.

H.3 CORRELATION BETWEEN W AND ΔW

See Figure 8 for the normalized subspace similarity between W and ΔW with varying r .

Note again that ΔW does not contain the top singular directions of W , since the similarity between the top 4 directions in ΔW and the top-10% of those in W barely exceeds 0.2. This gives evidence that ΔW contains those “task-specific” directions that are otherwise *not* emphasized in W .

An interesting next question to answer, is how “strong” do we need to amplify those task-specific directions, in order for the model adaptation to work well?

超参数	适应	MNLI-100	MNLI-1k	MNLI-10K	MNLI-392K
优化器	-	AdamW			
预热令牌	-	250000			
学习率调度	-	线性			
批大小	-	20	20	100	128
# 轮数	-	40	40	4	2
学习率	微调	5.00E-6			
	前缀嵌入	2.00E-04	2.00E-04	4.00E-04	5.00E-04
	前缀层 LoRA	5.00E-05	5.00E-05	5.00E-05	1.00E-04
适应-特定	LoRA	2.00E-4			
	前缀嵌入 l_p	16	32	64	256
	前缀嵌入 l_i	8			
	前缀微调	$l_p = l_i = 8$			
	LoRA	$r_q = r_v = 8$			

表 17：用于 MNLI(m)- n 不同 GPT-3 适应方法的超参数

其中我们的相似度定义为：

$$\phi(A, B, i, j) = \psi(U_A^i, U_B^j) = \frac{\sum_{i=1}^p \sigma_i^2}{p} = \frac{1}{p} \left(1 - d(U_A^i, U_B^j)^2 \right)$$

这种相似性满足：如果 U_A^i 和 U_B^j 共享相同的列跨度，那么 $\phi(A, B, i, j) = 1$ 。如果它们完全正交，那么 $\phi(A, B, i, j) = 0$ 。否则， $\phi(A, B, i, j) \in (0, 1)$ 。

H 低秩矩阵的附加实验

我们展示了我们关于低秩更新矩阵的研究的补充结果。

H.1 LoRA与LORA的相关性

LES

参见图6和图7，了解图3和图4中的结果如何推广到其他层。

H.2 r 对GPT-2的影响

我们在GPT-2中重复了关于 r （第7.2节）的实验。以E2E自然语言生成挑战数据集为例，我们报告了在训练26,000步后，不同 r 选择所达到的验证损失和测试指标。我们在表18中展示了我们的结果。GPT-2中型的最优秩取决于所使用的指标，在4到16之间，这与GPT-3 175B的情况类似。请注意，模型大小与适应最优秩之间的关系仍然是一个悬而未决的问题。

H.3 W 与 ΔW 的相关性

参见图8，了解 W 与 ΔW 在变化 r 下的归一化子空间相似度。

再次注意， ΔW 不包含 W 的前几个奇异方向，因为 ΔW 中的前4个方向与 W 中前10%的方向之间的相似度几乎不超过0.2。这表明 ΔW 包含了那些“任务特定”的方向，而这些方向在其他情况下并未被强调。

接下来一个有趣的问题是如何“强”地放大那些特定任务的指导方向，以便模型适应能够良好工作？

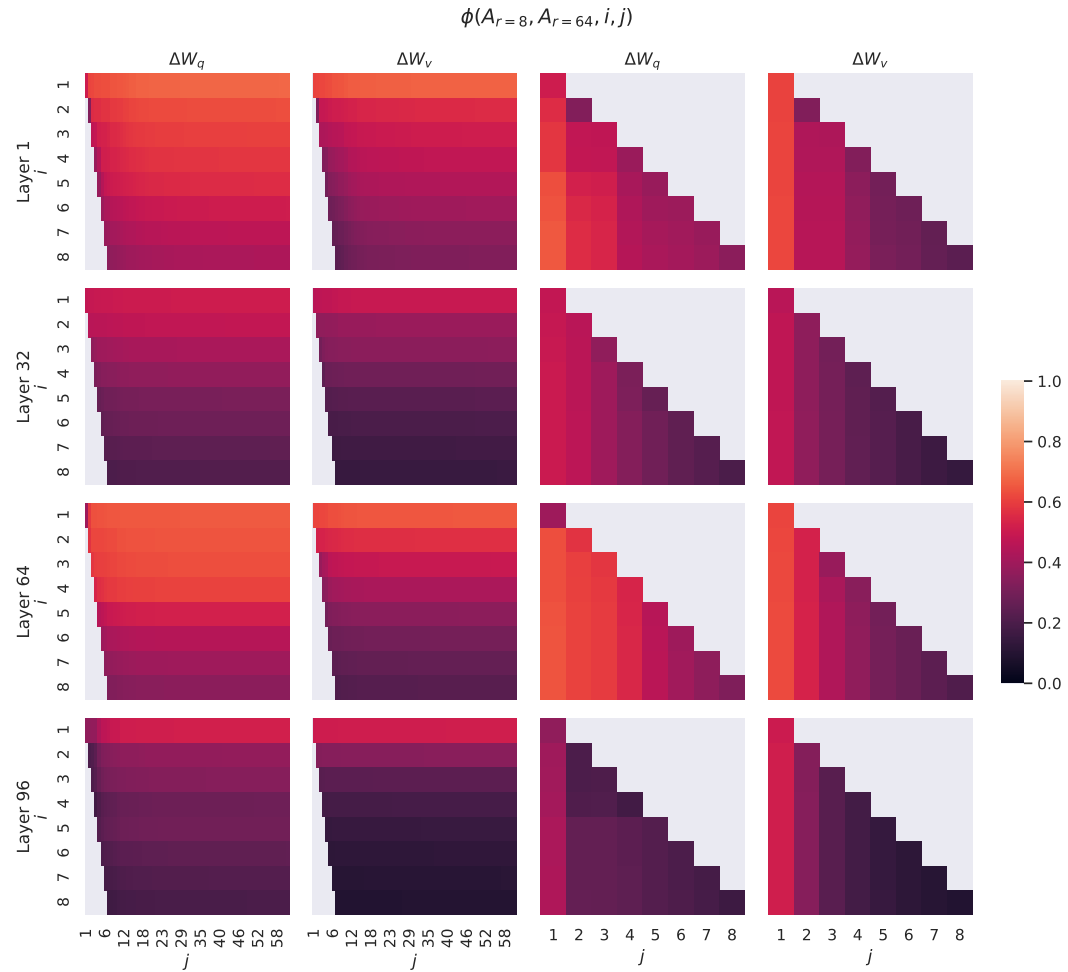


Figure 6: Normalized subspace similarity between the column vectors of $A_{r=8}$ and $A_{r=64}$ for both ΔW_q and ΔW_v from the 1st, 32nd, 64th, and 96th layers in a 96-layer Transformer.

H.4 AMPLIFICATION FACTOR

One can naturally consider a *feature amplification factor* as the ratio $\frac{\|\Delta W\|_F}{\|U^\top W V^\top\|_F}$, where U and V are the left- and right-singular matrices of the SVD decomposition of ΔW . (Recall $UU^\top W V^\top V$ gives the “projection” of W onto the subspace spanned by ΔW .)

Intuitively, when ΔW mostly contains task-specific directions, this quantity measures how much of them are amplified by ΔW . As shown in Section 7.3, for $r = 4$, this amplification factor is as large as 20. In other words, there are (generally speaking) four feature directions in each layer (out of the entire feature space from the pre-trained model W), that need to be amplified by a very large factor 20, in order to achieve our reported accuracy for the downstream specific task. And, one should expect a very different set of feature directions to be amplified for each different downstream task.

One may notice, however, for $r = 64$, this amplification factor is only around 2, meaning that *most* directions learned in ΔW with $r = 64$ are *not* being amplified by much. This should not be surprising, and in fact gives evidence (once again) that the intrinsic rank *needed* to represent the “task-specific directions” (thus for model adaptation) is low. In contrast, those directions in the rank-4 version of ΔW (corresponding to $r = 4$) are amplified by a much larger factor 20.

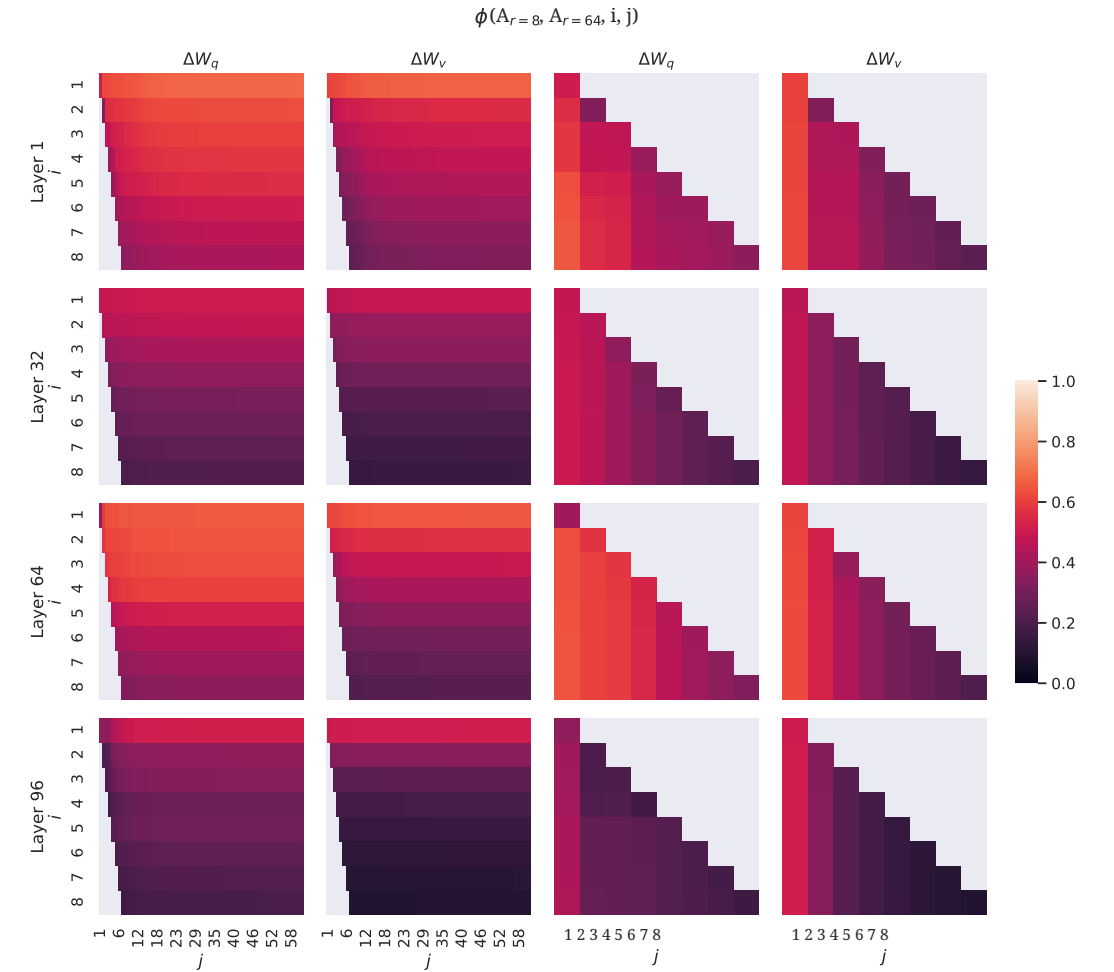


图6: 在96层Transformer模型中, 第1层、第32层、第64层和第96层中, $A_{r=8}$ 和 $A_{r=64}$ 的列向量之间的归一化子空间相似度。

H.4 放大因子

人们自然会考虑一个特征放大因子作为比率 $\frac{\|\Delta W\|_F}{\|U^\top W V^\top\|_F}$, 其中 U 和 V 是 ΔW 的 SVD 分解的左、右奇异矩阵。(回想 $UU^\top W V^\top V$ 给出了 W 在由 ΔW 张成的子空间上的“投影”。)

直观上, 当 ΔW 主要包含任务特定的方向时, 这个量度衡量了它们被 ΔW 放大了多少。如第 7.3 节所示, 对于 $r = 4$, 这个放大因子高达 20。换句话说, 为了达到我们报告的下游特定任务的准确率, 每个层 (在整个预训练模型的特征空间中) 通常有四个特征方向需要被放大一个非常大的因子 20。并且, 对于每个不同的下游任务, 应该期望被放大的特征方向集是截然不同的。

然而, 人们可能会注意到, 对于 $r = 64$, 这个放大因子只有大约 2, 这意味着在 ΔW 中使用 $r = 64$ 学习的大多数方向并没有被显著放大。这并不令人意外, 实际上也再次证明了代表“任务特定方向” (因此用于模型适应) 所需的秩是很低的。相比之下, 在 ΔW 的秩-4 版本 (对应于 $r = 4$) 中的那些方向则被放大了 20 倍这个大得多得多的因子。

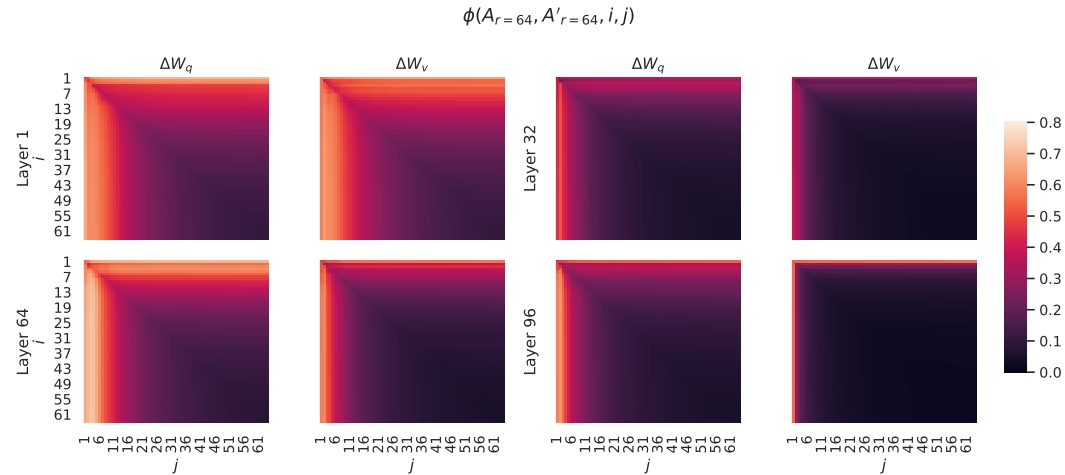


Figure 7: Normalized subspace similarity between the column vectors of $A_{r=64}$ from two randomly seeded runs, for both ΔW_q and ΔW_v from the 1st, 32nd, 64th, and 96th layers in a 96-layer Transformer.

Rank r	val_loss	BLEU	NIST	METEOR	ROUGE_L	CIDEr
1	1.23	68.72	8.7215	0.4565	0.7052	2.4329
2	1.21	69.17	8.7413	0.4590	0.7052	2.4639
4	1.18	70.38	8.8439	0.4689	0.7186	2.5349
8	1.17	69.57	8.7457	0.4636	0.7196	2.5196
16	1.16	69.61	8.7483	0.4629	0.7177	2.4985
32	1.16	69.33	8.7736	0.4642	0.7105	2.5255
64	1.16	69.24	8.7174	0.4651	0.7180	2.5070
128	1.16	68.73	8.6718	0.4628	0.7127	2.5030
256	1.16	68.92	8.6982	0.4629	0.7128	2.5012
512	1.16	68.78	8.6857	0.4637	0.7128	2.5025
1024	1.17	69.37	8.7495	0.4659	0.7149	2.5090

Table 18: Validation loss and test set metrics on E2E NLG Challenge achieved by LoRA with different rank r using GPT-2 Medium. Unlike on GPT-3 where $r = 1$ suffices for many tasks, here the performance peaks at $r = 16$ for validation loss and $r = 4$ for BLEU, suggesting the GPT-2 Medium has a similar intrinsic rank for adaptation compared to GPT-3 175B. Note that some of our hyperparameters are tuned on $r = 4$, which matches the parameter count of another baseline, and thus might not be optimal for other choices of r .

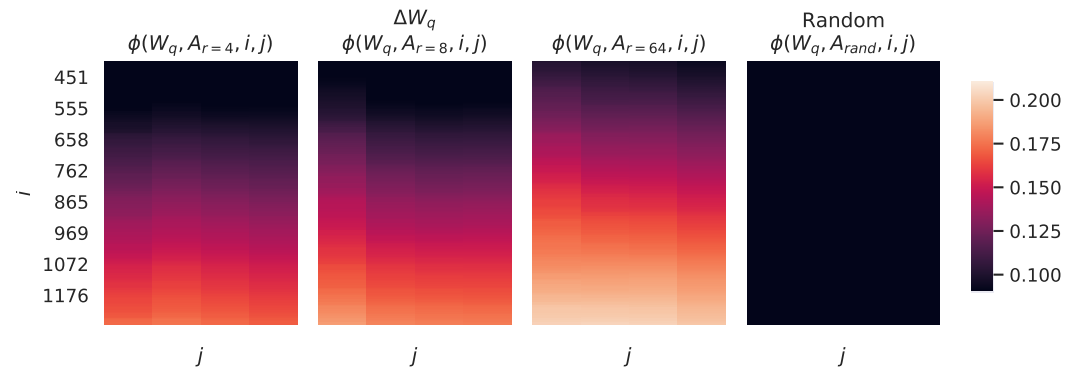


Figure 8: Normalized subspace similarity between the singular directions of W_q and those of ΔW_q with varying r and a random baseline. ΔW_q amplifies directions that are important but not emphasized in W . ΔW with a larger r tends to pick up more directions that are already emphasized in W .

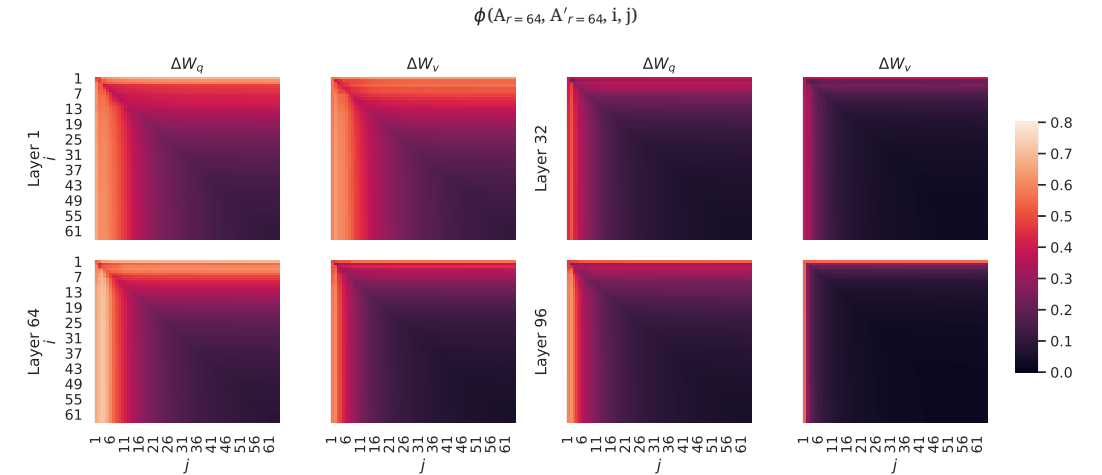


图7：两个随机初始化运行中来自 $A_{r=64}$ 的列向量之间的归一化子空间相似度，包括96层Transformer中的第1层、第32层、第64层和第96层的 ΔW_q 和 ΔW_v 。

秩 r	验证损失	BLEU	NIST	METEOR	ROUGE_L	CIDEr
1	1.23	68.72	8.7215	0.4565	0.7052	2.4329
2	1.21	69.17	8.7413	0.4590	0.7052	2.4639
4	1.18	70.38	8.8439	0.4689	0.7186	2.5349
8	1.17	69.57	8.7457	0.4636	0.7196	2.5196
16	1.16	69.61	8.7483	0.4629	0.7177	2.4985
32	1.16	69.33	8.7736	0.4642	0.7105	2.5255
64	1.16	69.24	8.7174	0.4651	0.7180	2.5070
128	1.16	68.73	8.6718	0.4628	0.7127	2.5030
256	1.16	68.92	8.6982	0.4629	0.7128	2.5012
512	1.16	68.78	8.6857	0.4637	0.7128	2.5025
1024	1.17	69.37	8.7495	0.4659	0.7149	2.5090

表18：LoRA使用不同秩 r 的GPT-2 Medium在端到端自然语言生成挑战中实现的验证损失和测试集指标。与GPT-3不同，在GPT-3中 $r = 1$ 足以完成许多任务，而在这里，验证损失在 $r = 16$ 处达到峰值，BLEU在 $r = 4$ 处达到峰值，这表明GPT-2 Medium在适应方面的内在秩与GPT-3 175B相似。请注意，我们的部分超参数是在 $r = 4$ 上调整的，这与另一个基线的参数数量匹配，因此可能不适用于其他 r 选择。

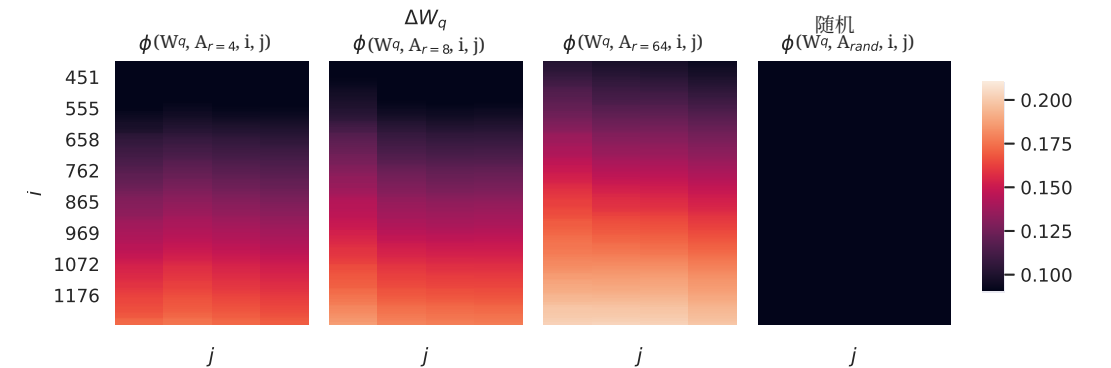


图8： W_q 的奇异方向与具有不同 r 和随机基线的 ΔW_q 的奇异方向之间的归一化子空间相似度。 ΔW_q 放大了在 W 中重要但未强调的方向。 ΔW 具有更大的 r 时，倾向于拾取在 W 中已经强调的更多方向。